

boba_extension_dim10

Dimension 10: Extension UI/UX Design Patterns - Comprehensive Research

Date: 2025-07-28 **Scope:** Browser extension UI/UX patterns for Boba torrent helper extension (Manifest V3) **Target:** Chrome/Edge (Chromium-based browsers)

Table of Contents

1. [Popup Page Design](#)
 2. [Options Page](#)
 3. [Context Menus](#)
 4. [chrome.action API](#)
 5. [Notifications](#)
 6. [Keyboard Shortcuts](#)
 7. [Side Panel \(Chrome 114+\)](#)
 8. [Extension Icon States](#)
 9. [Dark/Light Theme Support](#)
 10. [Responsive Design](#)
 11. [Accessibility](#)
 12. [Internationalization \(i18n\)](#)
-

1. Popup Page Design

1.1 Design Constraints & Best Practices

Claim: Chrome enforces hard limits on popup dimensions: 800x600px maximum, 25x25px minimum. The sweet spot for most extensions is 380-450px wide.

Source: Extension Booster / Chromium Source Code

URL: <https://extensionbooster.net/blog/chrome-extension-popup-ui-design-best-practices-guide/>

Date: 2026-04-23

Excerpt: "Chrome enforces hard limits on popup dimensions: 800x600px maximum, 25x25px minimum. But those limits aren't design targets -- they're walls... The sweet spot that top extensions actually use is 400x500px."

Context: Documented in Chromium source as ``static constexpr gfx::Size kMaxSize = {800, 600};``

Confidence: high

Claim: Popup should use CSS reset, explicit body width (not 100%), and `overflow-y: auto` for scrolling. System font stack is recommended.

Source: Reintech Chrome Extension UI Guide

URL: <https://reintech.io/blog/building-user-interfaces-chrome-extensions>

Date: 2026-03-24

Excerpt: "A typical popup should be 300-400px wide and 400-600px tall. Going wider can feel awkward, and taller popups may get cut off on smaller screens."

Context: Use system-ui font family for native feel across platforms

Confidence: high

1.2 Popup HTML Structure

```
<!-- popup/popup.html -->
<!DOCTYPE html>
<html lang="en" data-theme="auto">
<head>
  <meta charset="UTF-8" />
  <meta name="color-scheme" content="light dark" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Boba Torrent Helper</title>
  <link rel="stylesheet" href="popup.css" />
</head>
<body>
  <div id="app">
    <!-- Header: Fixed, non-scrolling -->
    <header class="popup-header">
```

```

<div class="header-left">
  
  <span class="header-title" data-i18n="popupTitle">Boba</
    span>
</div>
<div class="header-actions">
  <button id="refresh-btn" class="icon-btn" aria-
    label="Refresh torrents" title="Scan page">
    <svg width="16" height="16" viewBox="0 0 16 16"
      fill="none" aria-hidden="true">
      <path d="M13.5 8A5.5 5.5 0 0 1 3.3 10.5M2.5 8a5.5 5.5
        0 0 1 10.2-2.5M11.5 2.5V5h-2.5M4.5 13.5V11h2.5"
        stroke="currentColor" stroke-width="1.5" stroke-
        linecap="round" stroke-linejoin="round"/>
    </svg>
  </button>
  <button id="settings-btn" class="icon-btn" aria-
    label="Open settings" title="Settings">
    <svg width="16" height="16" viewBox="0 0 16 16"
      fill="none" aria-hidden="true">
      <circle cx="8" cy="8" r="2.5" stroke="currentColor"
        stroke-width="1.5"/>
      <path d="M12.8 8a4.8 4.8 0 0
        0-.26-1.57l1.48-1.48-1.06-1.06-1.48 1.48A4.8 4.8 0 0 0 9.9
        4.48V3h-1.5v1.48a4.8 4.8 0 0 0-1.57.26L5.35 3.26 4.29
        4.32l1.48 1.48A4.8 4.8 0 0 0 5.5 7.37H4v1.5h1.48c.07.54.22
        1.06.44 1.57l-1.48 1.48 1.06 1.06 1.48-1.48c.51.22 1.03.37
        1.57.44V15h1.5v-1.48c.54-.07 1.06-.22 1.57-.44l1.48 1.48
        1.06-1.06-1.48-1.48c.22-.51.37-1.03.44-1.57H15v-1.5h-1.48z"
        stroke="currentColor" stroke-width="1.5"/>
    </svg>
  </button>
</div>
</header>

<!-- Connection Status Bar -->
<div id="connection-status" class="connection-bar"
  role="status" aria-live="polite">
  <span class="status-dot" id="status-dot"></span>
  <span class="status-text" id="status-text">Connecting...</
    span>
</div>

<!-- Main Content: Scrollable -->
<main class="popup-content" id="main-content">
  <!-- Empty state (shown when no torrents detected) -->
  <div id="empty-state" class="empty-state">
    <svg width="48" height="48" viewBox="0 0 48 48"
      fill="none" class="empty-icon" aria-hidden="true">
      <path d="M24 4v8M24 36v8M4 24h8M36 24h8"
        stroke="currentColor" stroke-width="2" stroke-
        linecap="round"/>
      <circle cx="24" cy="24" r="8" stroke="currentColor"
        stroke-width="2"/>
    </div>

```

```

</svg>
<p class="empty-title" data-i18n="noTorrentsFound">No
torrents found</p>
<p class="empty-subtitle" data-i18n="scanThisPage">Click
refresh to scan this page</p>
</div>

<!-- Torrent List (hidden initially) -->
<div id="torrent-list-container" class="torrent-container"
style="display: none;">
  <div class="torrent-list-header">
    <label class="select-all-label">
      <input type="checkbox" id="select-all" aria-
label="Select all torrents" />
      <span id="torrent-count" data-i18n="torrentsFound">0
torrents</span>
    </label>
    <button id="send-selected" class="btn btn-primary btn-
sm" disabled>
      <span data-i18n="sendToBoba">Send to Boba</span>
    </button>
  </div>

  <ul id="torrent-list" class="torrent-list" role="listbox"
aria-label="Detected torrents">
    <!-- Dynamically populated -->
  </li></ul>

  <button id="send-all" class="btn btn-primary btn-full"
data-i18n="sendAllToBoba">
    Send All to Boba
  </button>
</div>

<!-- Sending progress state -->
<div id="sending-state" class="sending-state"
style="display: none;">
  <div class="progress-ring" role="progressbar" aria-
valuemin="0" aria-valuemax="100" aria-valuenow="0" aria-
label="Sending torrents">
    <svg class="progress-ring-svg" viewBox="0 0 100 100">
      <circle class="progress-track" cx="50" cy="50"
r="40" />
      <circle class="progress-fill" cx="50" cy="50" r="40"
id="progress-circle" />
    </svg>
    <span class="progress-percent" id="progress-
percent">0%</span>
  </div>
  <p class="sending-text" data-
i18n="sendingTorrents">Sending to Boba...</p>
  <p class="sending-detail" id="sending-detail"></p>
</div>

```

```

<!-- Success state -->
<div id="success-state" class="success-state"
  style="display: none;">
  <div class="success-icon">
    <svg width="48" height="48" viewBox="0 0 48 48"
      fill="none">
      <circle cx="24" cy="24" r="20" stroke="#10B981"
        stroke-width="2"/>
      <path d="M16 24l6 10-12" stroke="#10B981" stroke-
        width="2.5" stroke-linecap="round" stroke-
        linejoin="round"/>
    </svg>
  </div>
  <p class="success-title" data-
    i18n="sentSuccessfully">Sent successfully!</p>
  <p class="success-detail" id="success-detail"></p>
  <button id="view-dashboard-btn" class="btn btn-secondary
    btn-full" data-i18n="viewInDashboard">
    View in Dashboard
  </button>
</div>
</main>

<!-- Footer: Fixed -->
<footer class="popup-footer">
  <span class="footer-version">Boba v1.0</span>
  <a href="#" id="open-dashboard" class="footer-link" data-
    i18n="openDashboard">Dashboard</a>
</footer>
</div>
<script src="popup.js" type="module"></script>
</body>
</html>

```

1.3 Popup CSS (Theme-Aware)

```

/* popup/popup.css */

/* === CSS Reset === */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

/* === CSS Variables === */
:root {
  /* Light mode (default) */
  --bg-primary: #ffffff;
  --bg-secondary: #f8f9fa;
  --bg-tertiary: #e8eae5;
  --text-primary: #202124;

```

```

--text-secondary: #5f6368;
--text-tertiary: #80868b;
--border-color: #dadce0;
--accent-color: #1a73e8;
--accent-hover: #1557b0;
--accent-light: #d2e3fc;
--success-color: #10b981;
--error-color: #ef4444;
--warning-color: #f59e0b;
--shadow: 0 1px 3px rgba(60, 64, 67, 0.08);
--radius: 8px;
--radius-sm: 4px;

/* Sizing */
--popup-width: 400px;
--header-height: 48px;
--footer-height: 36px;
--connection-height: 28px;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg-primary: #202124;
    --bg-secondary: #292a2d;
    --bg-tertiary: #3c4043;
    --text-primary: #e8eaed;
    --text-secondary: #9aa0a6;
    --text-tertiary: #80868b;
    --border-color: #3c4043;
    --accent-color: #8ab4f8;
    --accent-hover: #aecbfa;
    --accent-light: #1a1c1e;
    --shadow: 0 1px 3px rgba(0, 0, 0, 0.3);
  }
}

/* === Base Styles === */
html, body {
  width: var(--popup-width);
  min-height: 400px;
  max-height: 580px;
  overflow: hidden; /* scroll happens in main */
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI',
    Roboto, Oxygen, Ubuntu, sans-serif;
  font-size: 14px;
  line-height: 1.5;
  color: var(--text-primary);
  background: var(--bg-primary);
}

```

```
#app {
  display: flex;
  flex-direction: column;
  height: 100vh;
  max-height: 580px;
}

/* === Header === */
.popup-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  height: var(--header-height);
  padding: 0 16px;
  background: var(--bg-secondary);
  border-bottom: 1px solid var(--border-color);
  flex-shrink: 0;
}

.header-left {
  display: flex;
  align-items: center;
  gap: 8px;
}

.header-icon {
  border-radius: 4px;
}

.header-title {
  font-size: 15px;
  font-weight: 600;
  color: var(--text-primary);
}

.header-actions {
  display: flex;
  gap: 4px;
}

.icon-btn {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 32px;
  height: 32px;
  border: none;
  border-radius: var(--radius-sm);
  background: transparent;
  color: var(--text-secondary);
  cursor: pointer;
}
```

```

    transition: background 0.15s, color 0.15s;
}

.icon-btn:hover {
    background: var(--bg-tertiary);
    color: var(--text-primary);
}

.icon-btn:focus-visible {
    outline: 2px solid var(--accent-color);
    outline-offset: 1px;
}

/* === Connection Status Bar === */
.connection-bar {
    display: flex;
    align-items: center;
    gap: 8px;
    height: var(--connection-height);
    padding: 0 16px;
    background: var(--bg-secondary);
    border-bottom: 1px solid var(--border-color);
    font-size: 12px;
    color: var(--text-secondary);
    flex-shrink: 0;
}

.status-dot {
    width: 8px;
    height: 8px;
    border-radius: 50%;
    background: var(--warning-color);
    flex-shrink: 0;
    transition: background 0.3s;
}

.status-dot.connected { background: var(--success-color); }
.status-dot.disconnected { background: var(--error-color); }
.status-dot.connecting { background: var(--warning-color);
    animation: pulse 1.5s infinite; }

@keyframes pulse {
    0%, 100% { opacity: 1; }
    50% { opacity: 0.4; }
}

/* === Main Content === */
.popup-content {
    flex: 1;
    overflow-y: auto;
    overflow-x: hidden;
    padding: 12px;

```



```
}

.popup-content::-webkit-scrollbar {
  width: 6px;
}

.popup-content::-webkit-scrollbar-track {
  background: transparent;
}

.popup-content::-webkit-scrollbar-thumb {
  background: var(--bg-tertiary);
  border-radius: 3px;
}

/* === Empty State === */
.empty-state {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  padding: 40px 20px;
  text-align: center;
  color: var(--text-secondary);
}

.empty-icon {
  margin-bottom: 16px;
  color: var(--text-tertiary);
}

.empty-title {
  font-size: 15px;
  font-weight: 500;
  color: var(--text-primary);
  margin-bottom: 4px;
}

.empty-subtitle {
  font-size: 13px;
  color: var(--text-secondary);
}

/* === Torrent List === */
.torrent-container {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.torrent-list-header {
  display: flex;
```

```

    align-items: center;
    justify-content: space-between;
    padding: 4px 4px;
}

.select-all-label {
    display: flex;
    align-items: center;
    gap: 8px;
    font-size: 13px;
    color: var(--text-secondary);
    cursor: pointer;
}

.select-all-label input[type="checkbox"] {
    width: 16px;
    height: 16px;
    accent-color: var(--accent-color);
    cursor: pointer;
}

.torrent-list {
    list-style: none;
    display: flex;
    flex-direction: column;
    gap: 6px;
    max-height: 340px;
    overflow-y: auto;
}

/* Individual torrent item */
.torrent-item {
    display: flex;
    align-items: flex-start;
    gap: 10px;
    padding: 10px 12px;
    border-radius: var(--radius);
    background: var(--bg-secondary);
    border: 1px solid var(--border-color);
    transition: border-color 0.15s, box-shadow 0.15s;
    cursor: pointer;
}

.torrent-item:hover {
    border-color: var(--accent-color);
    box-shadow: var(--shadow);
}

.torrent-item:focus-within {
    outline: 2px solid var(--accent-color);
    outline-offset: -1px;
}

```

```
.torrent-item.selected {
  border-color: var(--accent-color);
  background: var(--accent-light);
}

.torrent-checkbox {
  margin-top: 2px;
  width: 16px;
  height: 16px;
  accent-color: var(--accent-color);
  flex-shrink: 0;
  cursor: pointer;
}

.torrent-info {
  flex: 1;
  min-width: 0;
}

.torrent-name {
  font-size: 13px;
  font-weight: 500;
  color: var(--text-primary);
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  display: block;
}

.torrent-meta {
  display: flex;
  gap: 8px;
  margin-top: 2px;
  font-size: 11px;
  color: var(--text-tertiary);
}

.torrent-size::before { content: attr(data-size); }
.torrent-type-magnet::before { content: 'MAGNET'; }
.torrent-type-torrent::before { content: '.TORRENT'; }

.torrent-type-magnet,
.torrent-type-torrent {
  padding: 1px 5px;
  border-radius: 3px;
  font-size: 10px;
  font-weight: 600;
  letter-spacing: 0.3px;
}

.torrent-type-magnet {
```

```

    background: #dbeafe;
    color: #1d4ed8;
}

.torrent-type-torrent {
    background: #d1fae5;
    color: #047857;
}

@media (prefers-color-scheme: dark) {
    .torrent-type-magnet { background: #1e3a5f; color: #93c5fd; }
    .torrent-type-torrent { background: #064e3b; color: #6ee7b7; }
}

/* === Buttons === */
.btn {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 6px;
    padding: 9px 16px;
    border: none;
    border-radius: var(--radius-sm);
    font-size: 13px;
    font-weight: 500;
    cursor: pointer;
    transition: all 0.15s ease;
}

.btn-primary {
    background: var(--accent-color);
    color: #ffffff;
}

.btn-primary:hover:not(:disabled) {
    background: var(--accent-hover);
}

.btn-primary:disabled {
    opacity: 0.5;
    cursor: not-allowed;
}

.btn-secondary {
    background: var(--bg-tertiary);
    color: var(--text-primary);
    border: 1px solid var(--border-color);
}

.btn-secondary:hover {
    background: var(--border-color);
}

```

```
.btn-sm {
  padding: 6px 12px;
  font-size: 12px;
}

.btn-full {
  width: 100%;
  margin-top: 8px;
}

/* === Progress Ring === */
.sending-state {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  padding: 40px 20px;
}

.progress-ring {
  position: relative;
  width: 100px;
  height: 100px;
}

.progress-ring-svg {
  width: 100%;
  height: 100%;
  transform: rotate(-90deg);
}

.progress-track {
  fill: none;
  stroke: var(--bg-tertiary);
  stroke-width: 6;
}

.progress-fill {
  fill: none;
  stroke: var(--accent-color);
  stroke-width: 6;
  stroke-linecap: round;
  stroke-dasharray: 251.33; /* 2 * PI * 40 */
  stroke-dashoffset: 251.33;
  transition: stroke-dashoffset 0.3s ease;
}

.progress-percent {
  position: absolute;
  top: 50%;
  left: 50%;
```

```
    transform: translate(-50%, -50%);
    font-size: 20px;
    font-weight: 600;
    color: var(--text-primary);
}

.sending-text {
    margin-top: 16px;
    font-size: 15px;
    font-weight: 500;
    color: var(--text-primary);
}

.sending-detail {
    margin-top: 4px;
    font-size: 13px;
    color: var(--text-secondary);
}

/* === Success State === */
.success-state {
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 40px 20px;
    text-align: center;
}

.success-icon {
    margin-bottom: 16px;
}

.success-title {
    font-size: 15px;
    font-weight: 600;
    color: var(--text-primary);
    margin-bottom: 4px;
}

.success-detail {
    font-size: 13px;
    color: var(--text-secondary);
    margin-bottom: 16px;
}

/* === Footer === */
.popup-footer {
    display: flex;
    align-items: center;
    justify-content: space-between;
    height: var(--footer-height);
    padding: 0 16px;
```

```

    background: var(--bg-secondary);
    border-top: 1px solid var(--border-color);
    font-size: 11px;
    color: var(--text-tertiary);
    flex-shrink: 0;
}

.footer-link {
    color: var(--accent-color);
    text-decoration: none;
    font-weight: 500;
}

.footer-link:hover {
    text-decoration: underline;
}

/* === Focus Styles === */
:focus-visible {
    outline: 2px solid var(--accent-color);
    outline-offset: 2px;
}

/* === Reduced Motion === */
@media (prefers-reduced-motion: reduce) {
    *, *::before, *::after {
        animation-duration: 0.01ms !important;
        animation-iteration-count: 1 !important;
        transition-duration: 0.01ms !important;
    }
}

/* === High Contrast === */
@media (prefers-contrast: high) {
    .torrent-item {
        border-width: 2px;
    }
    .btn {
        border: 2px solid currentColor;
    }
}

```

1.4 Popup JavaScript

```

// popup/popup.js - Manifest V3, type="module"

// ===== State =====
let detectedTorrents = [];
let selectedIds = new Set();
let isSending = false;

```

```

// ===== DOM Elements =====
const els = {
  emptyState: document.getElementById('empty-state'),
  torrentContainer: document.getElementById('torrent-list-
    container'),
  torrentList: document.getElementById('torrent-list'),
  torrentCount: document.getElementById('torrent-count'),
  selectAll: document.getElementById('select-all'),
  sendSelected: document.getElementById('send-selected'),
  sendAll: document.getElementById('send-all'),
  sendingState: document.getElementById('sending-state'),
  successState: document.getElementById('success-state'),
  successDetail: document.getElementById('success-detail'),
  progressCircle: document.getElementById('progress-circle'),
  progressPercent: document.getElementById('progress-percent'),
  sendingDetail: document.getElementById('sending-detail'),
  statusDot: document.getElementById('status-dot'),
  statusText: document.getElementById('status-text'),
  refreshBtn: document.getElementById('refresh-btn'),
  settingsBtn: document.getElementById('settings-btn'),
  viewDashboardBtn: document.getElementById('view-dashboard-btn'),
  openDashboard: document.getElementById('open-dashboard'),
};

// ===== Utility: i18n helper =====
function t(key, substitutions) {
  return chrome.i18n.getMessage(key, substitutions) || key;
}

// ===== Initialize =====
document.addEventListener('DOMContentLoaded', async () => {
  // Load i18n strings
  document.querySelectorAll('[data-i18n]').forEach(el => {
    const key = el.dataset.i18n;
    el.textContent = t(key);
  });

  // Check connection to Boba server
  await checkConnection();

  // Request torrent scan from content script
  await scanPage();

  // Set up event listeners
  setupEventListeners();
});

function setupEventListeners() {
  els.refreshBtn.addEventListener('click', scanPage);

  els.settingsBtn.addEventListener('click', () => {
    chrome.runtime.openOptionsPage();
  });
}

```



```

});

els.selectAll.addEventListener('change', (e) => {
  const checkboxes = els.torrentList.querySelectorAll('.torrent-
checkbox');
  checkboxes.forEach(cb => {
    cb.checked = e.target.checked;
    const id = cb.dataset.id;
    e.target.checked ? selectedIds.add(id) :
      selectedIds.delete(id);
  });
  updateTorrentItems();
  updateSendButton();
});

els.sendSelected.addEventListener('click', () =>
  sendTorrents(Array.from(selectedIds)));
els.sendAll.addEventListener('click', () => {
  const allIds = detectedTorrents.map(t => t.id);
  sendTorrents(allIds);
});

els.viewDashboardBtn.addEventListener('click', openDashboard);
els.openDashboard.addEventListener('click', (e) => {
  e.preventDefault();
  openDashboard();
});
}

// ===== Connection Check =====
async function checkConnection() {
  try {
    const settings = await
      chrome.storage.sync.get('bobaServerUrl');
    const url = settings.bobaServerUrl || 'http://localhost:8080';

    updateConnectionStatus('connecting', t('connecting'));

    const response = await fetch(`${url}/api/health`, {
      method: 'GET',
      signal: AbortSignal.timeout(5000)
    });

    if (response.ok) {
      updateConnectionStatus('connected', t('connected'));
    } else {
      updateConnectionStatus('disconnected', t('serverError'));
    }
  } catch (err) {
    updateConnectionStatus('disconnected', t('notConnected'));
  }
}

```

```

function updateConnectionStatus(state, text) {
  els.statusDot.className = 'status-dot';
  els.statusDot.classList.add(state);
  els.statusText.textContent = text;
}

// ===== Page Scanning =====
async function scanPage() {
  updateConnectionStatus('connecting', t('scanning'));

  try {
    const [tab] = await chrome.tabs.query({ active: true,
      currentWindow: true });
    if (!tab?.id) return;

    // Inject content script if not already injected
    const results = await chrome.scripting.executeScript({
      target: { tabId: tab.id },
      func: () => {
        // Scan for magnet links and .torrent links
        const magnets =
          Array.from(document.querySelectorAll('a[href^="magnet:"]'))
            .map(a => ({
              type: 'magnet',
              url: a.href,
              name: a.textContent.trim() || a.href.substring(0, 60)
            + '...',
              source: window.location.href
            }));

        const torrents =
          Array.from(document.querySelectorAll('a[href$=".torrent"]'))
            .map(a => ({
              type: 'torrent',
              url: a.href,
              name: a.textContent.trim() || a.href.split('/').pop(),
              source: window.location.href
            }));

        // Also check for common torrent tracker patterns in page
        text
        const magnetMatches = document.body.innerText.match(/
          magnet:\?xt=urn:[a-z0-9]+:[a-z0-9]+/gi) || [];

        return { magnets, torrents, magnetTextMatches:
          magnetMatches.length };
      }
    });

    const scanResult = results[0]?.result || { magnets: [],
      torrents: [], magnetTextMatches: 0 };

```

```

// Merge and deduplicate
const all = [...scanResult.magnets, ...scanResult.torrents];
detectedTorrents = all.map((item, idx) => ({ ...item, id:
  `torrent-${idx}` }));

renderTorrentList();

// Update badge via background
await chrome.runtime.sendMessage({
  type: 'UPDATE_BADGE',
  count: detectedTorrents.length
});

} catch (err) {
  console.error('Scan failed:', err);
  showEmptyState(t('scanError'), t('tryAgain'));
}
}

// ===== Render Torrent List =====
function renderTorrentList() {
  if (detectedTorrents.length === 0) {
    els.emptyState.style.display = 'flex';
    els.torrentContainer.style.display = 'none';
    return;
  }

  els.emptyState.style.display = 'none';
  els.torrentContainer.style.display = 'flex';
  els.torrentCount.textContent = t('torrentsFound',
    [String(detectedTorrents.length)]);

  els.torrentList.innerHTML = detectedTorrents.map(torrent => `
    <li class="torrent-item" data-id="$
      {torrent.id}" role="option" tabindex="0">
      <input type="checkbox" class="torrent-checkbox" data-id="$
        {torrent.id}"
        ${selectedIds.has(torrent.id) ? 'checked' : ''}
        aria-label="Select ${escapeHtml(torrent.name)}" />
      <div class="torrent-info">
        <span class="torrent-name" title="$
          {escapeHtml(torrent.url)}">${escapeHtml(torrent.name)}</
          span>
        <div class="torrent-meta">
          <span class="torrent-type-${torrent.type}"></span>
          ${torrent.size ? `<span class="torrent-size" data-
            size="${torrent.size}"></span>` : ''}
        </div>
      </div>
    </li>
  `).join('');

  // Attach item-level event listeners

```

```

els.torrentList.querySelectorAll('.torrent-item').forEach(item
    => {
        const id = item.dataset.id;

        item.addEventListener('click', (e) => {
            if (e.target.classList.contains('torrent-checkbox')) return;
            const cb = item.querySelector('.torrent-checkbox');
            cb.checked = !cb.checked;
            toggleSelection(id, cb.checked);
        });

        item.addEventListener('keydown', (e) => {
            if (e.key === 'Enter' || e.key === ' ') {
                e.preventDefault();
                const cb = item.querySelector('.torrent-checkbox');
                cb.checked = !cb.checked;
                toggleSelection(id, cb.checked);
            }
        });
    });

els.torrentList.querySelectorAll('.torrent-
checkbox').forEach(cb => {
    cb.addEventListener('change', (e) => {
        toggleSelection(e.target.dataset.id, e.target.checked);
    });
});
}

function toggleSelection(id, isSelected) {
    isSelected ? selectedIds.add(id) : selectedIds.delete(id);
    updateTorrentItems();
    updateSendButton();
}

function updateTorrentItems() {
    els.torrentList.querySelectorAll('.torrent-item').forEach(item
        => {
            const id = item.dataset.id;
            item.classList.toggle('selected', selectedIds.has(id));
        });

    // Update select-all checkbox state
    const allCount = detectedTorrents.length;
    const checkedCount = selectedIds.size;
    els.selectAll.checked = checkedCount > 0;
    els.selectAll.indeterminate = checkedCount > 0 && checkedCount
        < allCount;
}

function updateSendButton() {
    els.sendSelected.disabled = selectedIds.size === 0 || isSending;
}

```

```

    const count = selectedIds.size;
    els.sendSelected.innerHTML = count > 0
      ? `${t('sendToBoba')} (${count})`
      : t('sendToBoba');
  }

function showEmptyState(title, subtitle) {
  els.emptyState.style.display = 'flex';
  els.emptyState.querySelector('.empty-title').textContent =
    title;
  els.emptyState.querySelector('.empty-subtitle').textContent =
    subtitle;
  els.torrentContainer.style.display = 'none';
}

function escapeHtml(text) {
  const div = document.createElement('div');
  div.textContent = text;
  return div.innerHTML;
}

// ===== Send Torrents =====
async function sendTorrents(ids) {
  if (isSending) return;
  isSending = true;

  const toSend = detectedTorrents.filter(t => ids.includes(t.id));

  // Show sending UI
  els.emptyState.style.display = 'none';
  els.torrentContainer.style.display = 'none';
  els.sendingState.style.display = 'flex';
  els.successState.style.display = 'none';

  updateProgress(0);

  try {
    const settings = await
      chrome.storage.sync.get(['bobaServerUrl', 'apiToken']);
    const url = settings.bobaServerUrl || 'http://localhost:8080';
    const token = settings.apiToken || '';

    for (let i = 0; i < toSend.length; i++) {
      const torrent = toSend[i];
      const percent = Math.round(((i + 0.5) / toSend.length) *
        100);
      updateProgress(percent);
      els.sendingDetail.textContent = t('sendingItem', [String(i
        + 1), String(toSend.length),
        escapeHtml(torrent.name.substring(0, 40))]));

      await fetch(`${url}/api/torrents/add`, {
        method: 'POST',

```

```

        headers: {
            'Content-Type': 'application/json',
            ...(token && { 'Authorization': `Bearer ${token}` })
        },
        body: JSON.stringify({
            url: torrent.url,
            name: torrent.name,
            type: torrent.type,
            source: torrent.source
        }),
        signal: AbortSignal.timeout(15000)
    });

    updateProgress(Math.round(((i + 1) / toSend.length) * 100));
}

// Show success
els.sendingState.style.display = 'none';
els.successState.style.display = 'flex';
els.successDetail.textContent = t('sentDetail',
    [String(toSend.length)]);

// Show notification
chrome.runtime.sendMessage({
    type: 'SHOW_NOTIFICATION',
    title: t('sentTitle'),
    message: t('sentMessage', [String(toSend.length)])
});

} catch (err) {
    console.error('Send failed:', err);
    els.sendingDetail.textContent = t('sendError');
    els.sendingDetail.style.color = 'var(--error-color)';
} finally {
    isSending = false;
}
}

function updateProgress(percent) {
    const circumference = 2 * Math.PI * 40; // r=40
    const offset = circumference - (percent / 100) * circumference;
    els.progressCircle.style.strokeDashoffset = offset;
    els.progressPercent.textContent = `${percent}%`;
    els.progressCircle.parentElement.setAttribute('aria-valuenow',
        String(percent));
}

function openDashboard() {
    chrome.storage.sync.get('bobaServerUrl', (result) => {
        const url = result.bobaServerUrl || 'http://localhost:8080';
        chrome.tabs.create({ url });
    });
}

```

```
});  
}
```

2. Options Page

2.1 Options Page HTML

```
<!-- options/options.html -->  
<!DOCTYPE html>  
<html lang="en" data-theme="auto">  
<head>  
  <meta charset="UTF-8" />  
  <meta name="color-scheme" content="light dark" />  
  <meta name="viewport" content="width=device-width, initial-  
    scale=1.0" />  
  <title>Boba Settings</title>  
  <link rel="stylesheet" href="options.css" />  
</head>  
<body>  
  <div class="options-container">  
    <header class="options-header">  
        
      <h1 data-il8n="optionsTitle">Boba Settings</h1>  
    </header>  
  
    <main class="options-content">  
      <!-- Server Connection Section -->  
      <section class="options-section" aria-labelledby="server-  
        heading">  
        <h2 id="server-heading" class="section-title">  
          <span class="section-icon">  
            <svg width="20" height="20" viewBox="0 0 24 24"  
              fill="none" stroke="currentColor" stroke-width="2">  
              <rect x="2" y="2" width="20" height="8" rx="2"/>  
              <rect x="2" y="14" width="20" height="8" rx="2"/>  
              <line x1="6" y1="6" x2="6" y2="6"/>  
              <line x1="6" y1="18" x2="6" y2="18"/>  
            </svg>  
          </span>  
          <span data-il8n="serverConnection">Server Connection</  
            span>  
        </h2>  
        <p class="section-desc" data-il8n="serverConnectionDesc">  
          Configure your Boba server URL and API credentials.  
        </p>  
  
        <div class="form-group">
```

```

    <label for="server-url" class="form-label" data-
i18n="serverUrlLabel">Boba Server URL</label>
    <input
      type="url"
      id="server-url"
      class="form-input"
      placeholder="http://localhost:8080"
      required
      aria-describedby="server-url-help"
    />
    <p id="server-url-help" class="form-help" data-
i18n="serverUrlHelp">
      The URL where your Boba server is running. Include the
      port if not using standard ports.
    </p>
  </div>

  <div class="form-group">
    <label for="api-token" class="form-label" data-
i18n="apiTokenLabel">API Token (Optional)</label>
    <div class="input-group">
      <input
        type="password"
        id="api-token"
        class="form-input"
        placeholder="Enter your API token"
        aria-describedby="api-token-help"
      />
      <button type="button" id="toggle-token" class="input-
btn" aria-label="Show token">
        <svg width="18" height="18" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2">
          <path d="M1 12s4-8 11-8 11 8 11 8-4 8-11
8-11-8-11-8z"/>
          <circle cx="12" cy="12" r="3"/>
        </svg>
      </button>
    </div>
    <p id="api-token-help" class="form-help" data-
i18n="apiTokenHelp">
      If your Boba server requires authentication, enter
      your API token here.
    </p>
  </div>

  <div class="form-actions">
    <button type="button" id="test-connection" class="btn
btn-secondary">
      <span data-i18n="testConnection">Test Connection</
span>
    </button>
    <span id="connection-result" class="connection-result"
role="status" aria-live="polite"></span>
  </div>

```


</section>

<!-- Download Settings Section -->

<section class="options-section" aria-labelledby="download-heading">

<h2 id="download-heading" class="section-title">

<svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">

<path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/>

>

<polyline points="7 10 12 15 17 10"/>

<line x1="12" y1="15" x2="12" y2="3"/>

</svg>

Download Settings

</h2>

<div class="form-group">

<label class="form-label" data-il8n="defaultCategory">Default Category</label>

<select id="default-category" class="form-select">

<option value="" data-il8n="categoryNone">None</option>

<option value="movies" data-il8n="categoryMovies">Movies</option>

<option value="tv" data-il8n="categoryTV">TV Shows</option>

<option value="music" data-il8n="categoryMusic">Music</option>

<option value="software" data-il8n="categorySoftware">Software</option>

<option value="other" data-il8n="categoryOther">Other</option>

</select>

</div>

<div class="form-group">

<label class="form-label toggle-label">

<input type="checkbox" id="auto-start" class="toggle-input" />

Auto-start downloads

</label>

<p class="form-help" data-il8n="autoStartHelp">

Automatically start downloading torrents when sent to Boba.

</p>

</div>

<div class="form-group">

<label class="form-label toggle-label">

```

        <input type="checkbox" id="notifications"
class="toggle-input" checked />
        <span class="toggle-switch" aria-hidden="true"></span>
        <span class="toggle-text" data-
il8n="enableNotifications">Enable notifications</span>
    </label>
    <p class="form-help" data-il8n="notificationsHelp">
        Show browser notifications for download events.
    </p>
</div>
</section>

```

```

<!-- Behavior Section -->

```

```

<section class="options-section" aria-labelledby="behavior-
heading">
    <h2 id="behavior-heading" class="section-title">
        <span class="section-icon">
            <svg width="20" height="20" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2">
                <circle cx="12" cy="12" r="3"/>
                <path d="M19.4 15a1.65 1.65 0 0 0 .33 1.82l.06.06a2
2 0 0 1 0 2.83 2 2 0 0 1-.283 .06-.06a1.65 1.65 0 0
0-1.82-.33 1.65 1.65 0 0 0-1 1.51V21a2 2 0 0 1-2 2 2 2 0 0
1-2-2v-.09A1.65 1.65 0 0 0 9 19.4a1.65 1.65 0 0
0-1.82.33l-.06.06a2 2 0 0 1-2.83 0 2 2 0 0 1
0-2.83l.06-.06a1.65 1.65 0 0 0 .33-1.82 1.65 1.65 0 0
0-1.51-1H3a2 2 0 0 1-2-2 2 2 0 0 1 2-2h.09A1.65 1.65 0 0
4.6 9a1.65 1.65 0 0 0-.33-1.82l-.06-.06a2 2 0 0 1 0-2.83 2
2 0 0 1 2.83 .06.06a1.65 1.65 0 0 0 1.82.33H9a1.65 1.65
0 0 0 1-1.51V3a2 2 0 0 1 2-2 2 2 0 0 1 2 2v.09a1.65 1.65 0
0 0 1 1.51 1.65 1.65 0 0 0 1.82-.33l.06-.06a2 2 0 0 1 2.83
0 2 2 0 0 1 0 2.83l-.06.06a1.65 1.65 0 0 0-.33 1.82V9a1.65
1.65 0 0 0 1.51 1H21a2 2 0 0 1 2 2 2 2 0 0 1-2 2h-.09a1.65
1.65 0 0 0-1.51 1z"/>
            </svg>
        </span>
        <span data-il8n="behavior">Behavior</span>
    </h2>

```

```

<div class="form-group">
    <label class="form-label" data-il8n="scanMode">Scan
Mode</label>
    <select id="scan-mode" class="form-select">
        <option value="manual" data-il8n="scanManual">Manual
(click to scan)</option>
        <option value="auto" data-il8n="scanAuto">Automatic
(on page load)</option>
    </select>
</div>

```

```

<div class="form-group">
    <label class="form-label toggle-label">
        <input type="checkbox" id="context-menu"
class="toggle-input" checked />

```

```

        <span class="toggle-switch" aria-hidden="true"></span>
        <span class="toggle-text" data-
i18n="contextMenu">Show context menu items</span>
      </label>
    </div>
  </section>

  <!-- Danger Zone -->
  <section class="options-section danger-section" aria-
labelledby="danger-heading">
    <h2 id="danger-heading" class="section-title danger-
title" data-i18n="dangerZone">Danger Zone</h2>
    <button type="button" id="reset-settings" class="btn btn-
danger">
      <span data-i18n="resetSettings">Reset All Settings</
span>
    </button>
  </section>
</main>

<footer class="options-footer">
  <button type="button" id="save-settings" class="btn btn-
primary btn-lg">
    <span data-i18n="saveSettings">Save Settings</span>
  </button>
  <span id="save-status" class="save-status" role="status"
aria-live="polite"></span>
</footer>
</div>
<script src="options.js" type="module"></script>
</body>
</html>

```

2.2 Options Page CSS

```

/* options/options.css */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

:root {
  --bg-primary: #ffffff;
  --bg-secondary: #f8f9fa;
  --text-primary: #202124;
  --text-secondary: #5f6368;
  --text-tertiary: #80868b;
  --border-color: #dadce0;
  --accent: #1a73e8;
  --accent-hover: #1557b0;
  --success: #10b981;

```

```

    --error: #ef4444;
    --danger: #dc2626;
    --radius: 8px;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg-primary: #202124;
    --bg-secondary: #292a2d;
    --text-primary: #e8eaed;
    --text-secondary: #9aa0a6;
    --text-tertiary: #80868b;
    --border-color: #3c4043;
    --accent: #8ab4f8;
    --accent-hover: #aecbfa;
    --danger: #ef4444;
  }
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI',
    Roboto, sans-serif;
  font-size: 14px;
  line-height: 1.5;
  color: var(--text-primary);
  background: var(--bg-secondary);
  min-height: 100vh;
}

.options-container {
  max-width: 640px;
  margin: 0 auto;
  padding: 24px 16px;
}

/* Header */
.options-header {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-bottom: 24px;
  padding-bottom: 16px;
  border-bottom: 1px solid var(--border-color);
}

.options-header h1 {
  font-size: 22px;
  font-weight: 600;
}

/* Sections */
.options-section {

```

```
    background: var(--bg-primary);
    border: 1px solid var(--border-color);
    border-radius: var(--radius);
    padding: 24px;
    margin-bottom: 16px;
}

.section-title {
    display: flex;
    align-items: center;
    gap: 8px;
    font-size: 16px;
    font-weight: 600;
    margin-bottom: 8px;
}

.section-desc {
    font-size: 13px;
    color: var(--text-secondary);
    margin-bottom: 16px;
}

.section-icon {
    color: var(--text-tertiary);
}

/* Form Elements */
.form-group {
    margin-bottom: 16px;
}

.form-group:last-child {
    margin-bottom: 0;
}

.form-label {
    display: block;
    font-size: 13px;
    font-weight: 500;
    color: var(--text-primary);
    margin-bottom: 6px;
}

.form-input,
.form-select {
    width: 100%;
    padding: 10px 12px;
    border: 1px solid var(--border-color);
    border-radius: var(--radius);
    background: var(--bg-primary);
    color: var(--text-primary);
    font-size: 14px;
}
```

```

    transition: border-color 0.15s, box-shadow 0.15s;
}

.form-input:focus,
.form-select:focus {
    outline: none;
    border-color: var(--accent);
    box-shadow: 0 0 0 3px rgba(26, 115, 232, 0.15);
}

@media (prefers-color-scheme: dark) {
    .form-input:focus,
    .form-select:focus {
        box-shadow: 0 0 0 3px rgba(138, 180, 248, 0.15);
    }
}

.form-help {
    font-size: 12px;
    color: var(--text-secondary);
    margin-top: 4px;
}

/* Input Group (for password show/hide) */
.input-group {
    display: flex;
    gap: 0;
}

.input-group .form-input {
    border-radius: var(--radius) 0 0 var(--radius);
    border-right: none;
}

.input-btn {
    padding: 10px 12px;
    border: 1px solid var(--border-color);
    border-radius: 0 var(--radius) var(--radius) 0;
    background: var(--bg-secondary);
    color: var(--text-secondary);
    cursor: pointer;
    transition: background 0.15s;
}

.input-btn:hover {
    background: var(--border-color);
}

/* Toggle Switch */
.toggle-label {
    display: flex !important;
    align-items: center;

```

```

    gap: 10px;
    cursor: pointer;
}

.toggle-input {
    position: absolute;
    opacity: 0;
    width: 0;
    height: 0;
}

.toggle-switch {
    width: 40px;
    height: 22px;
    background: var(--border-color);
    border-radius: 11px;
    position: relative;
    transition: background 0.2s;
    flex-shrink: 0;
}

.toggle-switch::after {
    content: '';
    position: absolute;
    width: 18px;
    height: 18px;
    border-radius: 50%;
    background: #fff;
    top: 2px;
    left: 2px;
    transition: transform 0.2s;
    box-shadow: 0 1px 3px rgba(0,0,0,0.2);
}

.toggle-input:checked + .toggle-switch {
    background: var(--accent);
}

.toggle-input:checked + .toggle-switch::after {
    transform: translateX(18px);
}

.toggle-input:focus-visible + .toggle-switch {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
}

.toggle-text {
    font-size: 13px;
}

/* Buttons */

```

```
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  padding: 10px 20px;
  border: none;
  border-radius: var(--radius);
  font-size: 14px;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.15s;
}

.btn-primary {
  background: var(--accent);
  color: #fff;
}

.btn-primary:hover {
  background: var(--accent-hover);
}

.btn-secondary {
  background: var(--bg-secondary);
  color: var(--text-primary);
  border: 1px solid var(--border-color);
}

.btn-secondary:hover {
  background: var(--border-color);
}

.btn-danger {
  background: transparent;
  color: var(--danger);
  border: 1px solid var(--danger);
}

.btn-danger:hover {
  background: var(--danger);
  color: #fff;
}

.btn-lg {
  padding: 12px 32px;
  font-size: 15px;
}

/* Danger Section */
.danger-section {
  border-color: var(--danger);
}
```



```

}

.danger-title {
  color: var(--danger);
}

/* Form Actions */
.form-actions {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-top: 16px;
}

.connection-result {
  font-size: 13px;
  font-weight: 500;
}

.connection-result.success { color: var(--success); }
.connection-result.error { color: var(--error); }

/* Footer */
.options-footer {
  display: flex;
  align-items: center;
  gap: 12px;
  padding-top: 8px;
}

.save-status {
  font-size: 13px;
  color: var(--text-secondary);
}

.save-status.success { color: var(--success); }

/* Focus */
:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 2px;
}

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    transition-duration: 0.01ms !important;
  }
}

```

2.3 Options Page JavaScript

```
// options/options.js - Manifest V3, type="module"

// ===== Default Settings =====
const DEFAULT_SETTINGS = {
  bobaServerUrl: 'http://localhost:8080',
  apiToken: '',
  defaultCategory: '',
  autoStart: false,
  notifications: true,
  scanMode: 'manual',
  contextMenu: true
};

// ===== DOM Elements =====
const els = {
  serverUrl: document.getElementById('server-url'),
  apiToken: document.getElementById('api-token'),
  toggleToken: document.getElementById('toggle-token'),
  testConnection: document.getElementById('test-connection'),
  connectionResult: document.getElementById('connection-result'),
  defaultCategory: document.getElementById('default-category'),
  autoStart: document.getElementById('auto-start'),
  notifications: document.getElementById('notifications'),
  scanMode: document.getElementById('scan-mode'),
  contextMenu: document.getElementById('context-menu'),
  resetSettings: document.getElementById('reset-settings'),
  saveSettings: document.getElementById('save-settings'),
  saveStatus: document.getElementById('save-status')
};

// ===== Load Settings =====
document.addEventListener('DOMContentLoaded', async () => {
  // Apply i18n
  document.querySelectorAll('[data-i18n]').forEach(el => {
    const key = el.dataset.i18n;
    const msg = chrome.i18n.getMessage(key);
    if (msg) el.textContent = msg;
  });

  // Load saved settings
  const stored = await
    chrome.storage.sync.get(Object.keys(DEFAULT_SETTINGS));
  const settings = { ...DEFAULT_SETTINGS, ...stored };

  // Apply to form
  applySettingsToForm(settings);

  // Set up listeners
  setupListeners();
});
```

```

    // Auto-save on change (debounced)
    setupAutoSave();
  });

  function applySettingsToForm(settings) {
    els.serverUrl.value = settings.bobaServerUrl || '';
    els.apiToken.value = settings.apiToken || '';
    els.defaultCategory.value = settings.defaultCategory || '';
    els.autoStart.checked = settings.autoStart;
    els.notifications.checked = settings.notifications;
    els.scanMode.value = settings.scanMode;
    els.contextMenu.checked = settings.contextMenu;
  }

  function getSettingsFromForm() {
    return {
      bobaServerUrl: els.serverUrl.value.trim() ||
        DEFAULT_SETTINGS.bobaServerUrl,
      apiToken: els.apiToken.value.trim(),
      defaultCategory: els.defaultCategory.value,
      autoStart: els.autoStart.checked,
      notifications: els.notifications.checked,
      scanMode: els.scanMode.value,
      contextMenu: els.contextMenu.checked
    };
  }

  function setupListeners() {
    // Toggle password visibility
    els.toggleToken.addEventListener('click', () => {
      const isPassword = els.apiToken.type === 'password';
      els.apiToken.type = isPassword ? 'text' : 'password';
      els.toggleToken.setAttribute('aria-label', isPassword ? 'Hide
        token' : 'Show token');
    });

    // Test Connection
    els.testConnection.addEventListener('click', async () => {
      els.connectionResult.textContent = '';
      const url = els.serverUrl.value.trim() ||
        DEFAULT_SETTINGS.bobaServerUrl;

      try {
        els.testConnection.disabled = true;
        els.connectionResult.textContent =
          chrome.i18n.getMessage('testing') || 'Testing...';
        els.connectionResult.className = 'connection-result';

        const response = await fetch(`${url}/api/health`, {
          signal: AbortSignal.timeout(8000)
        });
      }
    });
  }

```

```

    if (response.ok) {
      els.connectionResult.textContent =
        chrome.i18n.getMessage('connectionSuccess') ||
        'Connected!';
      els.connectionResult.className = 'connection-result
        success';
    } else {
      els.connectionResult.textContent =
        chrome.i18n.getMessage('connectionFailed') || 'Server
        error';
      els.connectionResult.className = 'connection-result
        error';
    }
  } catch (err) {
    els.connectionResult.textContent =
      chrome.i18n.getMessage('connectionError') || 'Cannot
        connect';
    els.connectionResult.className = 'connection-result error';
  } finally {
    els.testConnection.disabled = false;
  }
});

// Reset Settings
els.resetSettings.addEventListener('click', async () => {
  const confirmed =
    confirm(chrome.i18n.getMessage('resetConfirm') || 'Reset
      all settings?');
  if (confirmed) {
    await chrome.storage.sync.clear();
    applySettingsToForm(DEFAULT_SETTINGS);
    showSaveStatus('reset', 'Settings reset');
  }
});

// Save Button
els.saveSettings.addEventListener('click', saveSettings);
}

let saveTimeout;
function setupAutoSave() {
  // Auto-save 500ms after any input change
  const inputs = document.querySelectorAll('input, select,
    textarea');
  inputs.forEach(input => {
    input.addEventListener('change', () => {
      clearTimeout(saveTimeout);
      saveTimeout = setTimeout(saveSettings, 500);
    });
  });
});
}

```

```

async function saveSettings() {
  try {
    const settings = getSettingsFromForm();
    await chrome.storage.sync.set(settings);
    showSaveStatus('success', chrome.i18n.getMessage('saved') ||
      'Settings saved');

    // Notify background that settings changed
    chrome.runtime.sendMessage({ type: 'SETTINGS_CHANGED',
      settings });
  } catch (err) {
    console.error('Save failed:', err);
    showSaveStatus('error', chrome.i18n.getMessage('saveFailed')
      || 'Save failed');
  }
}

function showSaveStatus(type, message) {
  els.saveStatus.textContent = message;
  els.saveStatus.className = 'save-status';
  if (type === 'success') els.saveStatus.classList.add('success');

  setTimeout(() => {
    els.saveStatus.textContent = '';
  }, 3000);
}

```

2.4 Storage Architecture

Claim: chrome.storage.sync should be used for settings (100KB total, 8KB/item). chrome.storage.local for larger data.

chrome.storage.session for temporary MV3 state.

Source: Chrome Developer Docs

URL: <https://developer.chrome.com/docs/extensions/reference/api/storage>

Date: 2026-05-12

Excerpt: "storage.sync: data synced across devices, quota ~100KB

total, 8KB per item... storage.local: device-specific, 10MB

limit... storage.session: current browser session, 10MB, MV3 only"

Context: Use sync for user preferences, local for cached torrent lists, session for popup state

Confidence: high

Claim: The chrome.storage API preserves data types automatically - no JSON.parse/stringify needed. Supports monitoring changes via onChanged listener.

Source: Chrome Developer Docs / Reintech

URL: <https://developer.chrome.com/docs/extensions/reference/api/storage>

Date: 2026-05-12

Excerpt: "Unlike localStorage, chrome.storage preserves data types automatically... chrome.storage.onChanged.addListener((changes,

```
namespace) => { ... })"
```

Context: Background script listens to settings changes to reconfigure context menus, icon state

Confidence: high

3. Context Menu

3.1 Context Menu Registration

```
// background/contextMenus.js - Service Worker
```

```
const CONTEXT_MENU_ITEMS = {
  SEND_TO_BOBA_LINK: 'send-to-boba-link',
  SEND_TO_BOBA_PAGE: 'send-to-boba-page',
  SCAN_THIS_PAGE: 'scan-this-page',
  SEPARATOR_1: 'separator-1',
  OPEN_DASHBOARD: 'open-dashboard'
};

/**
 * Initialize all context menu items on install.
 * Also call this when settings change (e.g., context menu
 *   preference toggled).
 */
export async function initContextMenus() {
  // Clear existing
  await chrome.contextMenus.removeAll();

  // Check if context menus are enabled in settings
  const settings = await chrome.storage.sync.get('contextMenu');
  if (settings.contextMenu === false) return;

  // --- "Send to Boba" on links ---
  // Magnet links
  chrome.contextMenus.create({
    id: CONTEXT_MENU_ITEMS.SEND_TO_BOBA_LINK,
    title: chrome.i18n.getMessage('sendToBoba') || 'Send to Boba',
    contexts: ['link'],
    targetUrlPatterns: [
      'magnet:*',           // magnet links
      '*://*/*.torrent',    // .torrent file links
      '*://*/*.torrent?*'   // .torrent with query params
    ]
  });

  // --- "Scan this page" on page context ---
  chrome.contextMenus.create({
    id: CONTEXT_MENU_ITEMS.SCAN_THIS_PAGE,
```

```

        title: chrome.i18n.getMessage('scanThisPage') || 'Scan for
            torrents',
        contexts: ['page', 'action']
    });

    // --- Separator ---
    chrome.contextMenus.create({
        id: CONTEXT_MENU_ITEMS.SEPARATOR_1,
        type: 'separator',
        contexts: ['page', 'link']
    });

    // --- Open Dashboard ---
    chrome.contextMenus.create({
        id: CONTEXT_MENU_ITEMS.OPEN_DASHBOARD,
        title: chrome.i18n.getMessage('openDashboard') || 'Open Boba
            Dashboard',
        contexts: ['page', 'action']
    });
}

/**
 * Handle context menu clicks
 */
chrome.contextMenus.onClicked.addListener(async (info, tab) => {
    switch (info.menuItemId) {
        case CONTEXT_MENU_ITEMS.SEND_TO_BOBA_LINK:
            if (info.linkUrl) {
                await sendUrlToBoba(info.linkUrl, tab);
            }
            break;

        case CONTEXT_MENU_ITEMS.SCAN_THIS_PAGE:
            await chrome.tabs.sendMessage(tab.id, { type:
                'SCAN_REQUEST' });
            break;

        case CONTEXT_MENU_ITEMS.OPEN_DASHBOARD:
            const settings = await
                chrome.storage.sync.get('bobaServerUrl');
            const url = settings.bobaServerUrl || 'http://
                localhost:8080';
            chrome.tabs.create({ url });
            break;
    }
});

async function sendUrlToBoba(url, tab) {
    try {
        const settings = await
            chrome.storage.sync.get(['bobaServerUrl', 'apiToken',
                'defaultCategory', 'autoStart']);
    }

```

```

const serverUrl = settings.bobaServerUrl || 'http://
  localhost:8080';

// Determine type
const type = url.startsWith('magnet:') ? 'magnet' : 'torrent';

const response = await fetch(`${serverUrl}/api/torrents/add`,
  {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      ...(settings.apiToken && { 'Authorization': `Bearer $
        {settings.apiToken}` })
    },
    body: JSON.stringify({
      url,
      type,
      source: tab.url,
      category: settings.defaultCategory || undefined,
      autoStart: settings.autoStart ?? false
    })
  });

if (response.ok) {
  showNotification(
    chrome.i18n.getMessage('sentTitle') || 'Sent to Boba',
    chrome.i18n.getMessage('linkSent') || `Added: $
      {url.substring(0, 60)}...`
  );
  // Update badge
  const current = await chrome.action.getBadgeText({});
  const count = parseInt(current) || 0;
  await chrome.action.setBadgeText({ text: String(count +
    1) });
} else {
  showNotification(
    chrome.i18n.getMessage('sendFailed') || 'Failed to send',
    `Server returned ${response.status}`,
    'error'
  );
}
} catch (err) {
  console.error('Failed to send:', err);
  showNotification(
    chrome.i18n.getMessage('sendFailed') || 'Failed to send',
    err.message,
    'error'
  );
}
}

function showNotification(title, message, type = 'basic') {
  chrome.notifications.create({

```



```

    type: type === 'progress' ? 'progress' : 'basic',
    imageUrl: chrome.runtime.getURL('icons/icon-128.png'),
    title,
    message,
    ...(type === 'progress' ? { progress: 50 } : {})
  });
}

// Re-init context menus when settings change
chrome.storage.onChanged.addListener((changes, area) => {
  if (area === 'sync' && changes.contextMenu) {
    initContextMenus();
  }
});

// Also init on install/update
chrome.runtime.onInstalled.addListener(() => {
  initContextMenus();
});

```

3.2 Context Menu API Reference

Claim: Context menus require "contextMenus" permission in manifest. Items can target specific URL patterns via `targetUrlPatterns`. Types include "normal", "checkbox", "radio", "separator".

Source: Chrome Developer Docs

URL: <https://developer.chrome.com/docs/extensions/reference/api/contextMenus>

Date: 2026-05-15

Excerpt: "contexts: ['link'] with targetUrlPatterns: ['magnet:*', '*://*/*.torrent']... The onclick property is deprecated in MV3 event pages; use `chrome.contextMenus.onClicked.addListener` instead"

Context: MV3 best practice is to use the event listener pattern, not inline onclick

Confidence: high

Claim: If an extension creates more than one context menu item visible at once, Chrome automatically collapses them into a single parent menu labeled with the extension name.

Source: MDN Web Docs

URL: <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/menus>

Date: 2025-07-17

Excerpt: "If you have created more than one context menu item... then the items will be placed in a submenu. The submenu's parent will be labeled with the name of the extension."

Context: Keep menu items focused and minimal to avoid submenu

nesting
Confidence: high

4. chrome.action API

4.1 Badge & Icon State Management

```
// background/iconState.js - Service Worker

/**
 * Icon state definitions and management
 * States: idle, detecting, sending, error, connected,
 *         disconnected
 */

const ICON_STATES = {
  IDLE: {
    badgeText: '',
    badgeColor: [128, 128, 128, 255], // gray
    iconPath: {
      '16': 'icons/icon-16.png',
      '32': 'icons/icon-32.png',
      '48': 'icons/icon-48.png'
    },
    title: 'Boba - Click to scan'
  },
  DETECTING: {
    badgeText: '...',
    badgeColor: [245, 166, 35, 255], // orange
    iconPath: {
      '16': 'icons/icon-detecting-16.png',
      '32': 'icons/icon-detecting-32.png'
    },
    title: 'Boba - Scanning...'
  },
  FOUND: {
    badgeColor: [26, 115, 232, 255], // blue
    iconPath: {
      '16': 'icons/icon-16.png',
      '32': 'icons/icon-32.png'
    }
  },
  SENDING: {
    badgeText: '...',
    badgeColor: [245, 166, 35, 255], // orange
    iconPath: {
      '16': 'icons/icon-sending-16.png',
      '32': 'icons/icon-sending-32.png'
    },
  },
}
```

```

        title: 'Boba - Sending...'
    },
    ERROR: {
        badgeText: '!',
        badgeColor: [239, 68, 68, 255],    // red
        iconPath: {
            '16': 'icons/icon-error-16.png',
            '32': 'icons/icon-error-32.png'
        },
        title: 'Boba - Connection error'
    },
    CONNECTED: {
        badgeColor: [16, 185, 129, 255],    // green
        title: 'Boba - Connected'
    },
    DISCONNECTED: {
        badgeText: 'X',
        badgeColor: [239, 68, 68, 255],    // red
        iconPath: {
            '16': 'icons/icon-offline-16.png',
            '32': 'icons/icon-offline-32.png'
        },
        title: 'Boba - Disconnected'
    }
};

/**
 * Set the badge with detected torrent count
 */
export async function updateBadgeCount(count) {
    if (count === 0) {
        await chrome.action.setBadgeText({ text: '' });
        await chrome.action.setTitle({ title:
            ICON_STATES.IDLE.title });
    } else {
        // Badge text max ~4 chars
        const text = count > 999 ? '999+' : String(count);
        await chrome.action.setBadgeText({ text });
        await chrome.action.setBadgeBackgroundColor({ color:
            ICON_STATES.FOUND.badgeColor });
        await chrome.action.setTitle({ title: `Boba - ${count}
            torrents found` });
    }
}

/**
 * Set full icon state (for state-based icons)
 */
export async function setIconState(stateKey) {
    const state = ICON_STATES[stateKey];
    if (!state) return;

```

```

    if (state.badgeText !== undefined) {
      await chrome.action.setBadgeText({ text: state.badgeText });
    }
    if (state.badgeColor) {
      await chrome.action.setBadgeBackgroundColor({ color:
        state.badgeColor });
    }
    if (state.iconPath) {
      await chrome.action.setIcon({ path: state.iconPath });
    }
    if (state.title) {
      await chrome.action.setTitle({ title: state.title });
    }
  }
}

/**
 * Animate badge text for "detecting" state
 */
let detectingInterval;
export function startDetectingAnimation() {
  stopDetectingAnimation();
  setIconState('DETECTING');

  let dots = 0;
  detectingInterval = setInterval(() => {
    dots = (dots + 1) % 4;
    chrome.action.setBadgeText({ text: '.'.repeat(dots) || ' ' });
  }, 400);
}

export function stopDetectingAnimation() {
  if (detectingInterval) {
    clearInterval(detectingInterval);
    detectingInterval = null;
  }
}

/**
 * Handle incoming state update requests
 */
chrome.runtime.onMessage.addListener((message, sender,
  sendResponse) => {
  if (message.type === 'UPDATE_BADGE') {
    updateBadgeCount(message.count);
    return false;
  }
  if (message.type === 'SET_ICON_STATE') {
    setIconState(message.state);
    return false;
  }
  if (message.type === 'START_DETECTING') {
    startDetectingAnimation();
  }
});

```

```

        return false;
    }
    if (message.type === 'STOP_DETECTING') {
        stopDetectingAnimation();
        return false;
    }
});

// Optional: Dynamic icon generation via canvas for count-based
// badges
// This is more complex but produces better visuals:
export async function generateBadgeIcon(count) {
    const canvas = new OffscreenCanvas(128, 128);
    const ctx = canvas.getContext('2d');

    // Draw base icon (would load from image in practice)
    ctx.fillStyle = '#1a73e8';
    ctx.beginPath();
    ctx.arc(64, 64, 60, 0, Math.PI * 2);
    ctx.fill();

    // Draw count circle
    if (count > 0) {
        ctx.fillStyle = '#ef4444';
        ctx.beginPath();
        ctx.arc(96, 32, 28, 0, Math.PI * 2);
        ctx.fill();

        ctx.fillStyle = 'ffffff';
        ctx.font = 'bold 28px sans-serif';
        ctx.textAlign = 'center';
        ctx.textBaseline = 'middle';
        ctx.fillText(String(count > 99 ? 99 : count), 96, 32);
    }

    const imageData = ctx.getImageData(0, 0, 128, 128);
    await chrome.action.setIcon({ imageData });
}

```

4.2 Manifest Configuration

```

{
  "action": {
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png",
      "48": "icons/icon-48.png",
      "128": "icons/icon-128.png"
    },
    "default_title": "Boba Torrent Helper",
    "default_popup": "popup/popup.html"
  }
}

```

```
}  
}
```

4.3 API Reference

Claim: `chrome.action.setBadgeText` accepts a string (max ~4 chars visible). `setBadgeBackgroundColor` accepts RGBA array [0-255] or CSS color string. `setIcon` accepts either a path object or `ImageData`.

Source: Chrome Developer Docs

URL: <https://developer.chrome.com/docs/extensions/reference/api/action>

Date: 2025-08-11

Excerpt: "`chrome.action.setBadgeBackgroundColor({color: [0, 255, 0, 0]})... chrome.action.setBadgeText({text: '12'})... The text property is now required in the setBadge method"`

Context: MV3 requires text property; empty string '' clears the badge

Confidence: high

Claim: Badge text max ~4 characters displayed. Badge color can be set per-tab with `tabId` option (auto-resets on tab close).

Source: MDN Web Docs / Chrome Developer Docs

URL: <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/action/setBadgeBackgroundColor>

Date: 2025-11-06

Excerpt: "Tabs without a specific badge background color will inherit the global badge background color... If a `tabId` is specified, it removes the tab-specific badge"

Context: Tab-specific badges useful for showing per-page torrent counts

Confidence: high

5. Notifications

5.1 Notification Templates

// background/notifications.js - Service Worker

```
/**  
 * Notification templates for download lifecycle events  
 * Types: basic, image, list, progress  
 */
```

```
const NOTIFICATION_ICONS = {  
  default: chrome.runtime.getURL('icons/icon-128.png'),  
  success: chrome.runtime.getURL('icons/icon-success-128.png'),  
  error: chrome.runtime.getURL('icons/icon-error-128.png')  
}
```

```

};

/**
 * Show "Download Started" notification
 */
export function notifyDownloadStarted(torrentName, torrentId) {
  chrome.notifications.create(`started-${torrentId}`, {
    type: 'basic',
    imageUrl: NOTIFICATION_ICONS.default,
    title: chrome.i18n.getMessage('downloadStarted') || 'Download
      Started',
    message: truncate(torrentName, 80),
    buttons: [
      { title: chrome.i18n.getMessage('viewDashboard') || 'View
        Dashboard' }
    ],
    priority: 0
  });
}

/**
 * Show "Download Complete" notification
 */
export function notifyDownloadComplete(torrentName, torrentId) {
  chrome.notifications.create(`complete-${torrentId}`, {
    type: 'basic',
    imageUrl: NOTIFICATION_ICONS.success,
    title: chrome.i18n.getMessage('downloadComplete') ||
      'Download Complete',
    message: truncate(torrentName, 80),
    buttons: [
      { title: chrome.i18n.getMessage('openFile') || 'Open
        File' },
      { title: chrome.i18n.getMessage('showFolder') || 'Show
        Folder' }
    ],
    priority: 1
  });
}

/**
 * Show "Download Error" notification
 */
export function notifyDownloadError(torrentName,
  torrentId, error) {
  chrome.notifications.create(`error-${torrentId}`, {
    type: 'basic',
    imageUrl: NOTIFICATION_ICONS.error,
    title: chrome.i18n.getMessage('downloadError') || 'Download
      Error',
    message: `${truncate(torrentName, 50)}: ${truncate(error, 60)}
      `,
    priority: 2
  });
}

```

```

    });
}

/**
 * Show "Progress" notification (use sparingly - can be noisy)
 */
export function notifyProgress(torrentName, percent, torrentId) {
    // Only update if percent changed significantly (avoid spam)
    const key = `progress-${torrentId}`;
    chrome.notifications.create(key, {
        type: 'progress',
        imageUrl: NOTIFICATION_ICONS.default,
        title: chrome.i18n.getMessage('downloading') || 'Downloading',
        message: truncate(torrentName, 80),
        progress: Math.round(percent),
        priority: 0
    });
}

/**
 * Show "Batch Send" notification (when sending from popup)
 */
export function notifyBatchSent(count) {
    chrome.notifications.create('batch-sent', {
        type: 'basic',
        imageUrl: NOTIFICATION_ICONS.success,
        title: chrome.i18n.getMessage('sentTitle') || 'Sent to Boba',
        message: chrome.i18n.getMessage('batchSentMessage',
            [String(count)]) ||
            `${count} torrent(s) sent to Boba`,
        priority: 1
    });
}

/**
 * Show "Batch Results" list notification
 */
export function notifyBatchResults(results) {
    const items = results.slice(0, 5).map(r => ({
        title: r.success ? '✓' : '✗',
        message: truncate(r.name, 40)
    }));

    chrome.notifications.create('batch-results', {
        type: 'list',
        imageUrl: NOTIFICATION_ICONS.default,
        title: chrome.i18n.getMessage('sendResults') || 'Send
            Results',
        message: `${results.filter(r => r.success).length}/${
            results.length} successful`,
        items,
        priority: 1
    });
}

```



```

    });
}

// ===== Event Handlers =====

// Handle notification clicks
chrome.notifications.onClicked.addListener((notificationId) => {
    if (notificationId === 'batch-sent' ||
        notificationId.startsWith('started-')) {
        openDashboard();
    }
    // Close the notification
    chrome.notifications.clear(notificationId);
});

// Handle notification button clicks
chrome.notifications.onButtonClicked.addListener((notificationId,
    buttonIndex) => {
    if (notificationId.startsWith('complete-')) {
        const torrentId = notificationId.replace('complete-', '');
        if (buttonIndex === 0) {
            // Open file
            chrome.runtime.sendMessage({ type: 'OPEN_FILE',
                torrentId });
        } else if (buttonIndex === 1) {
            // Show folder
            chrome.runtime.sendMessage({ type: 'SHOW_FOLDER',
                torrentId });
        }
    }
    chrome.notifications.clear(notificationId);
});

// Handle notification close
chrome.notifications.onClosed.addListener((notificationId,
    byUser) => {
    // Clean up any tracking state if needed
});

// ===== Helpers =====
function truncate(str, maxLength) {
    return str.length > maxLength ? str.substring(0, maxLength - 3)
        + '...' : str;
}

function openDashboard() {
    chrome.storage.sync.get('bobaServerUrl', (result) => {
        const url = result.bobaServerUrl || 'http://localhost:8080';
        chrome.tabs.create({ url });
    });
}

```

5.2 Notification API Reference

Claim: chrome.notifications supports 4 template types: basic, image, list, progress. Buttons (up to 2) supported via buttons array. Events: onClicked, onButtonClicked, onClosed, onPermissionLevelChanged.

Source: Chrome Developer Docs

URL: <https://developer.chrome.com/docs/extensions/reference/api/notifications>

Date: 2026-03-26

Excerpt:

"chrome.notifications.onButtonClicked.addListener((notificationId, buttonIndex) => { ... })... onClicked fires when non-button area clicked"

Context: Progress type shows a progress bar (0-100). Image type includes imageUrl for preview.

Confidence: high

Claim: Platform differences exist: macOS Chrome 59+ shows native notifications instead of Chrome's custom UI. Images are not shown on macOS. List notifications only show first item on macOS.

Source: Chrome Developer Docs (Rich Notifications)

URL: <https://developer.chrome.com/docs/extensions/mv2/richNotifications>

Date: 2014-06-25

Excerpt: "For Mac OS X users on Chrome 59+, images are not shown... list notifications only display the first item"

Context: Design notifications with these limitations in mind for cross-platform support

Confidence: high

6. Keyboard Shortcuts

6.1 Manifest Definition

```
{
  "commands": {
    "send-all-torrents": {
      "suggested_key": {
        "default": "Ctrl+Shift+B",
        "mac": "Command+Shift+B"
      },
      "description": "Send all detected torrents to Boba"
    },
    "scan-page": {
      "suggested_key": {
        "default": "Ctrl+Shift+S",
        "mac": "Command+Shift+S"
      }
    }
  }
}
```

```

    },
    "description": "Scan current page for torrents"
  },
  "open-dashboard": {
    "suggested_key": {
      "default": "Ctrl+Shift+D",
      "mac": "Command+Shift+D"
    },
    "description": "Open Boba dashboard"
  },
  "toggle-side-panel": {
    "suggested_key": {
      "default": "Ctrl+Shift+P",
      "mac": "Command+Shift+P"
    },
    "description": "Open Boba side panel"
  },
  "_execute_action": {
    "suggested_key": {
      "default": "Ctrl+Shift+U",
      "mac": "Command+Shift+U"
    },
    "description": "Open Boba popup"
  }
}
}
}

```

6.2 Command Handler in Service Worker

// background/commands.js - Service Worker

```

chrome.commands.onCommand.addListener(async (command) => {
  console.log(`[Boba] Command received: ${command}`);

  switch (command) {
    case 'send-all-torrents':
      await handleSendAll();
      break;

    case 'scan-page':
      await handleScanPage();
      break;

    case 'open-dashboard':
      await handleOpenDashboard();
      break;

    case 'toggle-side-panel':
      await handleToggleSidePanel();
      break;
  }
}

```

```

});

async function handleSendAll() {
  try {
    const [tab] = await chrome.tabs.query({ active: true,
      currentWindow: true });
    if (!tab?.id) return;

    // Request torrent data from content script
    const results = await chrome.scripting.executeScript({
      target: { tabId: tab.id },
      func: () => {
        const magnets =
          Array.from(document.querySelectorAll('a[href^="magnet:"]')).map(a
            => a.href);
        const torrents =
          Array.from(document.querySelectorAll('a[href$=".torrent"]')).map(a
            => a.href);
        return [...magnets, ...torrents];
      }
    });

    const urls = results[0]?.result || [];
    if (urls.length === 0) {
      chrome.notifications.create({
        type: 'basic',
        iconUrl: chrome.runtime.getURL('icons/icon-128.png'),
        title: 'Boba',
        message: 'No torrents found on this page'
      });
      return;
    }

    // Send to server
    const settings = await
      chrome.storage.sync.get(['bobaServerUrl', 'apiToken']);
    const serverUrl = settings.bobaServerUrl || 'http://
      localhost:8080';

    let successCount = 0;
    for (const url of urls) {
      try {
        await fetch(`${serverUrl}/api/torrents/add`, {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
            ...(settings.apiToken && { 'Authorization': `Bearer $
              {settings.apiToken}` })
          },
          body: JSON.stringify({ url, type:
            url.startsWith('magnet:') ? 'magnet' : 'torrent' })
        });
        successCount++;
      }
    }
  }
}

```

```

    } catch (e) { /* individual failure, continue */ }
  }

  chrome.notifications.create({
    type: 'basic',
    imageUrl: chrome.runtime.getURL('icons/icon-
      success-128.png'),
    title: 'Boba - Sent',
    message: `${successCount}/${urls.length} torrents sent`
  });

} catch (err) {
  console.error('Send all failed:', err);
}
}

async function handleScanPage() {
  // Open popup (which triggers scan) or send message to active
  tab
  const [tab] = await chrome.tabs.query({ active: true,
    currentWindow: true });
  if (tab?.id) {
    chrome.tabs.sendMessage(tab.id, { type: 'SCAN_REQUEST' });
  }
}

async function handleOpenDashboard() {
  const settings = await chrome.storage.sync.get('bobaServerUrl');
  const url = settings.bobaServerUrl || 'http://localhost:8080';
  chrome.tabs.create({ url });
}

async function handleToggleSidePanel() {
  try {
    const [tab] = await chrome.tabs.query({ active: true,
      currentWindow: true });
    if (tab?.windowId) {
      await chrome.sidePanel.open({ windowId: tab.windowId });
    }
  } catch (err) {
    console.error('Side panel toggle failed:', err);
  }
}

```

6.3 API Reference

Claim: Commands API allows up to 4 suggested keys per extension. All combinations must include Ctrl or Alt. Special shortcuts: `_execute_action` (MV3), `_execute_browser_action` (MV2). Commands fire `chrome.commands.onCommand` in service worker.

Source: Chrome Developer Docs / MDN

URL: <https://developer.chrome.com/docs/extensions/reference/api/>

commands

Date: 2026-04-02

Excerpt: "_execute_action (Manifest V3) reserved for triggering action... These commands do not dispatch command.onCommand events like standard commands"

Context: _execute_action opens popup and does NOT fire onCommand - use standard commands for custom actions

Confidence: high

Claim: Supported keys: A-Z, 0-9, Comma, Period, Home, End, PageUp, PageDown, Space, Insert, Delete, Arrow keys, Media Keys. On Mac, 'Ctrl' maps to 'Command' by default; use 'MacCtrl' for actual Control key.

Source: Chrome Extension Docs

URL: <https://sunnyzhou-1024.github.io/chrome-extension-docs/apps/commands.html>

Date: N/A

Excerpt: "On Mac 'Ctrl' is automatically converted to 'Command'. If you want 'Ctrl' instead, please specify 'MacCtrl' under 'mac'"

Context: Critical for cross-platform shortcuts

Confidence: high

7. Side Panel (Chrome 114+)

7.1 Side Panel Implementation

```
<!-- sidepanel/sidepanel.html -->
<!DOCTYPE html>
<html lang="en" data-theme="auto">
<head>
  <meta charset="UTF-8" />
  <meta name="color-scheme" content="light dark" />
  <link rel="stylesheet" href="sidepanel.css" />
</head>
<body>
  <div class="sidepanel">
    <header class="sp-header">
      <div class="sp-brand">
        
        <span class="sp-title" data-il8n="sidePanelTitle">Boba Downloads</span>
      </div>
      <div class="sp-connection" id="sp-connection">
        <span class="sp-dot" id="sp-dot"></span>
        <span class="sp-status-text" id="sp-status-text">--</span>
      </div>
    </header>
```

```

<div class="sp-toolbar">
  <button id="sp-refresh" class="sp-btn-icon" aria-
    label="Refresh" title="Refresh">
    <svg width="14" height="14" viewBox="0 0 16 16"
      fill="none" stroke="currentColor" stroke-width="1.5">
      <path d="M13.5 8A5.5 5.5 0 0 1 3.3 10.5M2.5 8a5.5 5.5 0
        0 1 10.2-2.5M11.5 2.5V5h-2.5M4.5 13.5V11h2.5" stroke-
          linecap="round" stroke-linejoin="round"/>
      </svg>
    </button>
    <span class="sp-toolbar-divider"></span>
    <button id="sp-pause-all" class="sp-btn-icon" aria-
      label="Pause all" title="Pause all">
      <svg width="14" height="14" viewBox="0 0 16 16"
        fill="currentColor">
        <rect x="3" y="2" width="4" height="12" rx="1"/>
        <rect x="9" y="2" width="4" height="12" rx="1"/>
      </svg>
    </button>
    <button id="sp-resume-all" class="sp-btn-icon" aria-
      label="Resume all" title="Resume all">
      <svg width="14" height="14" viewBox="0 0 16 16"
        fill="currentColor">
        <path d="M4 2l10 6-10 6V2z"/>
      </svg>
    </button>
  </div>

  <main class="sp-content" id="sp-content">
    <!-- Empty state -->
    <div id="sp-empty" class="sp-empty">
      <p data-il8n="noActiveDownloads">No active downloads</p>
    </div>

    <!-- Download list -->
    <ul id="sp-download-list" class="sp-list" style="display:
      none;"></ul>
  </main>
</div>
<script src="sidepanel.js" type="module"></script>
</body>
</html>

```

```
// sidepanel/sidepanel.js
```

```
// Side panel has access to all Chrome APIs like popup
let refreshInterval;
```

```
// Notify service worker that side panel is ready
document.addEventListener('DOMContentLoaded', () => {
  chrome.runtime.sendMessage({ type: 'SIDE_PANEL_LOADED' });
});

```

```

document.getElementById('sp-refresh').addEventListener('click',
    refreshDownloads);
document.getElementById('sp-pause-
all').addEventListener('click', () =>
    controlAll('pause'));
document.getElementById('sp-resume-
all').addEventListener('click', () =>
    controlAll('resume'));

// Initial load and periodic refresh
refreshDownloads();
refreshInterval = setInterval(refreshDownloads, 3000);
});

window.addEventListener('beforeunload', () => {
    if (refreshInterval) clearInterval(refreshInterval);
});

async function refreshDownloads() {
    try {
        const settings = await
            chrome.storage.sync.get('bobaServerUrl');
        const url = settings.bobaServerUrl || 'http://localhost:8080';

        const response = await fetch(`${url}/api/torrents`, {
            signal: AbortSignal.timeout(5000)
        });
        const torrents = await response.json();

        renderDownloads(torrents);
        updateConnection(true, `${torrents.length} active`);
    } catch (err) {
        updateConnection(false, 'Disconnected');
    }
}

function renderDownloads(torrents) {
    const list = document.getElementById('sp-download-list');
    const empty = document.getElementById('sp-empty');

    if (torrents.length === 0) {
        list.style.display = 'none';
        empty.style.display = 'flex';
        return;
    }

    list.style.display = 'flex';
    empty.style.display = 'none';

    list.innerHTML = torrents.map(t => `
        <li class="sp-item" data-id="${t.id}">
            <div class="sp-item-header">

```



```

        <span class="sp-item-name" title="${escapeHtml(t.name)}">${
        escapeHtml(t.name)}</span>
        <span class="sp-item-status sp-status-${t.status}">${
        t.status}</span>
    </div>
    <div class="sp-item-progress">
        <div class="sp-progress-bar">
            <div class="sp-progress-fill" style="width: $
            {t.progress}%"></div>
        </div>
        <span class="sp-progress-text">${Math.round(t.progress)}
        %</span>
    </div>
    <div class="sp-item-meta">
        <span>${formatBytes(t.downloaded)} / $
        {formatBytes(t.size)}</span>
        <span>${formatSpeed(t.speed)}</span>
    </div>
</li>
`).join('');
}

function updateConnection(connected, text) {
    const dot = document.getElementById('sp-dot');
    const statusText = document.getElementById('sp-status-text');
    dot.className = 'sp-dot' + (connected ? ' connected' : '
    disconnected');
    statusText.textContent = text;
}

function formatBytes(b) {
    if (b === 0) return '0 B';
    const units = ['B', 'KB', 'MB', 'GB', 'TB'];
    const i = Math.floor(Math.log10(b) / 3);
    return (b / Math.pow(1024, i)).toFixed(1) + ' ' + units[i];
}

function formatSpeed(bps) {
    return formatBytes(bps) + '/s';
}

function escapeHtml(text) {
    const d = document.createElement('div');
    d.textContent = text;
    return d.innerHTML;
}

async function controlAll(action) {
    // Send pause/resume all command to server
    const settings = await chrome.storage.sync.get('bobaServerUrl');
    const url = settings.bobaServerUrl || 'http://localhost:8080';
    await fetch(`${url}/api/torrents/${action}all`, { method:
    'POST' });
}

```

```
    refreshDownloads();
}
```

7.2 Manifest & Service Worker Configuration

```
{
  "permissions": ["sidePanel"],
  "side_panel": {
    "default_path": "sidepanel/sidepanel.html"
  },
  "action": {
    "default_title": "Click to open Boba side panel"
  }
}

// background/sidePanel.js - Service Worker

import { initContextMenus } from './contextMenus.js';

// Configure action icon to toggle side panel
chrome.sidePanel.setPanelBehavior({ openPanelOnActionClick:
  true })
  .catch(err => console.error('Failed to set panel behavior:',
    err));

// Optional: Open side panel from context menu
// (see contextMenus.js for the menu item creation)

// Open side panel programmatically (requires user gesture
  context)
export async function openSidePanel(windowId) {
  try {
    await chrome.sidePanel.open({ windowId });
  } catch (err) {
    console.error('Failed to open side panel:', err);
  }
}

// Site-specific side panel (only show on torrent sites)
const TORRENT_SITE_PATTERNS = [
  'https://*.thepiratebay.*/*',
  'https://*.1337x.*/*',
  'https://*.rarbg.*/*',
  'https://*.nyaa.*/*'
];

chrome.tabs.onUpdated.addListener(async (tabId, info, tab) => {
  // Only proceed if we have a URL (page loaded)
  if (!tab.url) return;

  // Optional: Enable/disable side panel based on site
```

```
// By default, side panel is globally available
});
```

7.3 Side Panel API Reference

Claim: Side Panel API available in Chrome 114+, MV3 only. Requires "sidePanel" permission. Panel can be opened by: action click (setPanelBehavior), user gesture (sidePanel.open), context menu. Cannot auto-open programmatically without user interaction.

Source: Chrome Developer Docs

URL: <https://developer.chrome.com/docs/extensions/reference/api/sidePanel>

Date: 2026-02-17

Excerpt:

"chrome.sidePanel.setPanelBehavior({ openPanelOnActionClick: true })... sidePanel.open() may only be called in response to a user gesture"

Context: Side panel persists between tabs if configured as global. Tab-specific panels available.

Confidence: high

Claim: sidePanel.open() requires either tabId or windowId.

sidePanel.setOptions() enables/disables panel per-tab.

sidePanel.close() available in Chrome 141+.

Source: Chrome Developer Docs / W3C WebExtensions Issues

URL: <https://github.com/w3c/webextensions/issues/521>

Date: 2024-01-12

Excerpt: "The sidePanel can be opened via chrome.sidePanel.open() but the API isn't symmetrical, there is no .close() API"

Context: Workaround for closing: window.close() from within sidepanel.js, or setOptions({enabled: false}) from background

Confidence: high

8. Extension Icon States

8.1 Icon Set Requirements

Claim: Chrome extension icons should be provided in multiple sizes: 16x16 (favicon/menu), 32x32 (Windows high-DPI), 48x48 (Chrome Web Store), 128x128 (Chrome Web Store listing). Action toolbar uses 16, 24, 32.

Source: Chrome Developer Docs

URL: <https://developer.chrome.com/docs/extensions/reference/api/action>

Date: 2025-08-11

Excerpt: "action.default_icon: { '16': 'icon-16.png', '24': 'icon-24.png', '32': 'icon-32.png' }"

Context: Provide SVG for scalable icons where possible; Chrome

will rasterize appropriately
Confidence: high

8.2 Recommended Icon Set

Icon	Purpose	Sizes
icon-*.png	Default state	16, 32, 48, 128
icon-detecting-*.png	Scanning/loading state	16, 32
icon-sending-*.png	Sending to server	16, 32
icon-error-*.png	Connection error	16, 32
icon-offline-*.png	Server disconnected	16, 32
icon-success-*.png	Notification: success	128

8.3 Icon State Manager (Complete)

```
// background/iconManager.js

/**
 * Complete icon state manager with SVG-based dynamic icons
 * No external image files needed - generates icons
 * programmatically
 */

// Base Boba icon SVG path (a simple download arrow + circle)
const BOBA_PATH = 'M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10 10-4.48
  10-10S17.52 2 12 2zm-1 14.5v-4H8l4-4 4
  4h-3v4h-2zm1-13c4.14 0 7.5 3.36 7.5 7.5S16.14 19 12
  19s-7.5-3.36-7.5-7.5S7.86 4 12 4z';

function createIconCanvas(size, color, badge) {
  const canvas = new OffscreenCanvas(size, size);
  const ctx = canvas.getContext('2d');

  // Clear
  ctx.clearRect(0, 0, size, size);

  // Main icon circle
  ctx.fillStyle = color;
  ctx.beginPath();
  ctx.arc(size / 2, size / 2, size * 0.42, 0, Math.PI * 2);
  ctx.fill();

  // Inner "B" text or symbol
  ctx.fillStyle = '#ffffff';
  ctx.font = `bold ${size * 0.5}px sans-serif`;
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
```

```

ctx.fillText('B', size / 2, size / 2);

// Badge circle (top-right)
if (badge) {
  const badgeRadius = size * 0.22;
  const bx = size - badgeRadius - 1;
  const by = badgeRadius + 1;

  ctx.fillStyle = badge.color;
  ctx.beginPath();
  ctx.arc(bx, by, badgeRadius, 0, Math.PI * 2);
  ctx.fill();

  // Badge text
  ctx.fillStyle = 'ffffff';
  const fontSize = badge.text.length > 1 ? size * 0.2 : size *
    0.26;
  ctx.font = `bold ${fontSize}px sans-serif`;
  ctx.fillText(badge.text, bx, by + 1);
}

return ctx.getImageData(0, 0, size, size);
}

export async function setDynamicIcon(state, count) {
  const colors = {
    idle: '#5f6368',
    detecting: '#f59e0b',
    found: '#1a73e8',
    sending: '#f59e0b',
    error: '#ef4444',
    connected: '#10b981',
    disconnected: '#ef4444'
  };

  const color = colors[state] || colors.idle;

  const badge = count > 0 ? {
    color: state === 'error' ? '#ef4444' : '#1a73e8',
    text: count > 99 ? '99' : String(count)
  } : state === 'error' ? {
    color: '#ef4444',
    text: '!'
  } : null;

  const sizes = [16, 32];
  const imageData = {};

  for (const size of sizes) {
    imageData[size] = createIconCanvas(size, color, badge);
  }
}

```

```

await chrome.action.setIcon({ imageData });

// Also update badge text separately for accessibility
if (count > 0) {
  await chrome.action.setBadgeText({
    text: count > 999 ? '999+' : String(count)
  });
  await chrome.action.setBadgeBackgroundColor({ color });
} else {
  await chrome.action.setBadgeText({ text: '' });
}
}

```

9. Dark/Light Theme Support

9.1 Theme Detection Methods

Claim: Chrome extensions support dark mode via CSS media query `prefers-color-scheme` and meta color-scheme tag. Use CSS variables for dynamic theme switching. JavaScript can detect via `matchMedia()`.

Source: text/plain blog / Chrome Developer Blog

URL: <https://textslashplain.com/2021/10/17/spooky-enhancing-dark-mode-in-chromium/>

Date: 2022-08-03

Excerpt: "To indicate that a page supports dark mode styling, simply add a color-scheme meta tag: `<meta name='color-scheme' content='light dark'>`... use `prefers-color-scheme` media query"

Context: CSS variable approach updates dynamically when OS theme changes without page reload

Confidence: high

9.2 CSS Implementation

```

/* === Complete Theme-Aware CSS Framework === */

/* Base: Light mode */
:root {
  color-scheme: light dark;

  /* Backgrounds */
  --bg-primary: #ffffff;
  --bg-secondary: #f8f9fa;
  --bg-tertiary: #e8eaed;
  --bg-elevated: #ffffff;

  /* Text */
  --text-primary: #202124;

```

```

--text-secondary: #5f6368;
--text-tertiary: #80868b;
--text-on-accent: #ffffff;

/* Accent */
--accent: #1a73e8;
--accent-hover: #1557b0;
--accent-light: #d2e3fc;
--accent-alpha: rgba(26, 115, 232, 0.12);

/* Semantic */
--success: #10b981;
--success-bg: #d1fae5;
--error: #ef4444;
--error-bg: #fee2e2;
--warning: #f59e0b;
--warning-bg: #fef3c7;
--info: #3b82f6;

/* UI */
--border: #dadce0;
--border-hover: #bdc1c6;
--shadow-sm: 0 1px 2px rgba(60, 64, 67, 0.08);
--shadow: 0 1px 3px rgba(60, 64, 67, 0.12);
--shadow-lg: 0 4px 12px rgba(60, 64, 67, 0.15);

/* Spacing */
--radius-sm: 4px;
--radius: 8px;
--radius-lg: 12px;
}

/* Dark mode */
@media (prefers-color-scheme: dark) {
  :root {
    --bg-primary: #202124;
    --bg-secondary: #292a2d;
    --bg-tertiary: #3c4043;
    --bg-elevated: #35363a;

    --text-primary: #e8eaed;
    --text-secondary: #9aa0a6;
    --text-tertiary: #80868b;

    --accent: #8ab4f8;
    --accent-hover: #aecbfa;
    --accent-light: #1a1c1e;
    --accent-alpha: rgba(138, 180, 248, 0.12);

    --success: #34d399;
    --success-bg: #064e3b;
    --error: #f87171;
  }
}

```

```

    --error-bg: #450a0a;
    --warning: #fbbf24;
    --warning-bg: #451a03;

    --border: #3c4043;
    --border-hover: #5f6368;
    --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.3);
    --shadow: 0 1px 3px rgba(0, 0, 0, 0.4);
    --shadow-lg: 0 4px 12px rgba(0, 0, 0, 0.5);
  }
}

/* Force light/dark via data attribute (user override) */
[data-theme="light"] {
  color-scheme: light;
  /* ... override all dark variables to light values ... */
}

[data-theme="dark"] {
  color-scheme: dark;
  /* ... override all light variables to dark values ... */
}

/* Usage: just reference variables */
body {
  background: var(--bg-primary);
  color: var(--text-primary);
}

.btn-primary {
  background: var(--accent);
  color: var(--text-on-accent);
}

.btn-primary:hover {
  background: var(--accent-hover);
}

```

9.3 JavaScript Theme Detection

```

// utils/theme.js

/**
 * Detect and respond to theme changes
 */

export function isDarkMode() {
  return window.matchMedia('(prefers-color-scheme:
    dark)').matches;
}

```



```

export function onThemeChange(callback) {
  const mq = window.matchMedia('(prefers-color-scheme: dark)');
  mq.addEventListener('change', (e) => callback(e.matches));
  return () => mq.removeEventListener('change', callback);
}

// Listen for theme changes and update meta if needed
export function initThemeListener() {
  const mq = window.matchMedia('(prefers-color-scheme: dark)');

  // Set initial
  updateMetaThemeColor(mq.matches);

  // Listen for changes
  mq.addEventListener('change', (e) => {
    updateMetaThemeColor(e.matches);
  });
}

function updateMetaThemeColor(isDark) {
  // Update theme-color meta for mobile Chrome
  let meta = document.querySelector('meta[name="theme-color"]');
  if (!meta) {
    meta = document.createElement('meta');
    meta.name = 'theme-color';
    document.head.appendChild(meta);
  }
  meta.content = isDark ? '#202124' : '#ffffff';
}

// For user manual override
export function setTheme(theme) {
  document.documentElement.setAttribute('data-theme', theme);
  chrome.storage.sync.set({ theme });
}

export async function loadThemePreference() {
  const { theme } = await chrome.storage.sync.get('theme');
  if (theme === 'light' || theme === 'dark') {
    document.documentElement.setAttribute('data-theme', theme);
  }
  // 'auto' or undefined = follow system
}

```

10. Responsive Design

10.1 Popup Constraints

Claim: Chrome popup max size: 800x600px (Chromium source: gfx::Size kMaxSize = {800, 600}). Recommended popup width: 380-450px. Height should be content-driven with max-height and overflow-y: auto for scrolling.

Source: Extension Booster / Chromium Source

URL: <https://extensionbooster.net/blog/chrome-extension-popup-ui-design-best-practices-guide/>

Date: 2026-04-23

Excerpt: "The sweet spot that top extensions actually use is 400x500px. It fits comfortably in peripheral vision without covering the page content underneath."

Context: Define explicit width on both html and body elements. Never rely on width: 100%.

Confidence: high

10.2 Responsive CSS Patterns

```
/* popup/popup.css - Responsive additions */

/* Base popup sizing */
html, body {
  width: 400px;
  min-height: 300px;
  max-height: 580px;
  overflow: hidden;
}

/* Mobile-friendly touch targets */
button, .torrent-item, select, input[type="checkbox"] {
  min-height: 32px;
  min-width: 32px;
}

/* Scrollbar styling for consistency */
.popup-content::-webkit-scrollbar {
  width: 6px;
}

.popup-content::-webkit-scrollbar-thumb {
  background: var(--border-color);
  border-radius: 3px;
}

.popup-content::-webkit-scrollbar-thumb:hover {
  background: var(--text-tertiary);
}
```

```
/* Handle long text with ellipsis */
.torrent-name {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  max-width: 280px;
}

/* Flexbox-based responsive layout */
.torrent-list-header {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  align-items: center;
  justify-content: space-between;
}

/* Ensure footer doesn't get pushed off-screen */
#app {
  display: flex;
  flex-direction: column;
  height: 100vh;
  max-height: 580px;
}

.popup-content {
  flex: 1;
  min-height: 0; /* Critical: allows flex child to shrink below
                  content size */
  overflow-y: auto;
}

/* Density modes */
[data-density="compact"] .torrent-item {
  padding: 6px 10px;
}
[data-density="comfortable"] .torrent-item {
  padding: 14px 16px;
}

/* For high-DPI displays */
@media (-webkit-min-device-pixel-ratio: 2) {
  .header-icon,
  .torrent-checkbox {
    image-rendering: -webkit-optimize-contrast;
  }
}
```

11. Accessibility

11.1 ARIA Labels & Roles

Claim: All interactive elements must have accessible names. Use `aria-label` for icon-only buttons, `aria-labelledby` for grouped controls, `role` for custom widgets. Focus indicators must be visible.

Source: AllAccessible / Chrome Extension UI Guide

URL: <https://www.allaccessible.org/blog/implementing-aria-labels-for-web-accessibility>

Date: 2024-01-18

Excerpt: "aria-label: Provides accessible name for elements...
`role='dialog'`: Modal dialog... `aria-live='polite'`: Announce updates when convenient"

Context: Screen reader users need context for all interactive elements

Confidence: high

11.2 Accessibility Implementation

```
<!-- Key accessibility patterns -->
```

```
<!-- Icon-only buttons MUST have aria-label -->
```

```
<button class="icon-btn" aria-label="Send to Boba" title="Send to Boba">  
  <svg aria-hidden="true">...</svg>  
</button>
```

```
<!-- Live regions for dynamic content -->
```

```
<div id="status" role="status" aria-live="polite" class="sr-only"></div>
```

```
<!-- Listbox pattern for torrent list -->
```

```
<ul role="listbox" aria-label="Detected torrents" aria-multiselectable="true">  
  <li role="option" aria-selected="false" tabindex="0">...  
</ul>
```

```
<!-- Progress indicators with ARIA -->
```

```
<div role="progressbar"  
  aria-valuemin="0"  
  aria-valuemax="100"  
  aria-valuenow="42"  
  aria-label="Upload progress">  
  <div class="progress-bar"><div class="progress-fill"  
    style="width: 42%"></div></div>  
</div>
```

```
<!-- Ensure form labels are associated -->
```

```

<label for="server-url">Server URL</label>
<input id="server-url" type="url" aria-describedby="url-help" />
<p id="url-help">Enter the Boba server URL</p>

/* Accessibility additions */

/* Visually hidden (screen reader only) */
.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}

/* Visible focus indicators */
:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 2px;
}

/* Skip link for keyboard navigation */
.skip-link {
  position: absolute;
  top: -40px;
  left: 0;
  background: var(--accent);
  color: #fff;
  padding: 8px;
  text-decoration: none;
  z-index: 100;
}
.skip-link:focus {
  top: 0;
}

/* Ensure sufficient color contrast (WCAG AA: 4.5:1 for text) */
/* Our variable choices above meet this standard */

/* Reduced motion preference */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}

```

```

/* High contrast mode */
@media (prefers-contrast: high) {
  .torrent-item {
    border-width: 2px;
  }
  .btn {
    border: 2px solid currentColor;
  }
}

// utils/ally.js - Accessibility utilities

/**
 * Announce a message to screen readers
 */
export function announce(message, priority = 'polite') {
  let announcer = document.getElementById('ally-announcer');
  if (!announcer) {
    announcer = document.createElement('div');
    announcer.id = 'ally-announcer';
    announcer.className = 'sr-only';
    announcer.setAttribute('aria-live', priority);
    announcer.setAttribute('aria-atomic', 'true');
    document.body.appendChild(announcer);
  }

  // Clear and set (forces re-announcement)
  announcer.textContent = '';
  requestAnimationFrame(() => {
    announcer.textContent = message;
  });
}

/**
 * Trap focus within an element (modal dialog pattern)
 */
export function trapFocus(element) {
  const focusable = element.querySelectorAll(
    'button, [href], input, select, textarea, '
    '[tabindex]:not([tabindex="-1"])'
  );
  const first = focusable[0];
  const last = focusable[focusable.length - 1];

  element.addEventListener('keydown', (e) => {
    if (e.key !== 'Tab') return;

    if (e.shiftKey && document.activeElement === first) {
      e.preventDefault();
      last.focus();
    } else if (!e.shiftKey && document.activeElement === last) {
      e.preventDefault();
    }
  });
}

```

```

        first.focus();
    }
});
}

/**
 * Keyboard navigation for custom list
 */
export function setupListKeyboardNavigation(listElement, options
    = {}) {
    const items = () =>
        listElement.querySelectorAll('[role="option"]');

    listElement.addEventListener('keydown', (e) => {
        const current = document.activeElement;
        const itemList = Array.from(items());
        const index = itemList.indexOf(current);

        switch (e.key) {
            case 'ArrowDown':
                e.preventDefault();
                const next = itemList[index + 1] || itemList[0];
                next?.focus();
                break;
            case 'ArrowUp':
                e.preventDefault();
                const prev = itemList[index - 1] ||
                    itemList[itemList.length - 1];
                prev?.focus();
                break;
            case 'Home':
                e.preventDefault();
                itemList[0]?.focus();
                break;
            case 'End':
                e.preventDefault();
                itemList[itemList.length - 1]?.focus();
                break;
            case ' ':
                e.preventDefault();
                // Toggle selection
                current.click();
                break;
        }
    });
}

```

12. Internationalization (i18n)

12.1 Directory Structure

```
extension-root/  
  _locales/  
    en/  
      messages.json  
    es/  
      messages.json  
    fr/  
      messages.json  
    de/  
      messages.json  
    zh_CN/  
      messages.json  
  manifest.json  
  ...
```

12.2 messages.json (English - Complete Reference)

```
{  
  "extName": {  
    "message": "Boba Torrent Helper",  
    "description": "Extension display name"  
  },  
  "extDescription": {  
    "message":  
      "Detect and send torrents/magnet links to your Boba  
      server",  
    "description": "Extension description in store"  
  },  
  "popupTitle": {  
    "message": "Boba",  
    "description": "Popup header title"  
  },  
  "noTorrentsFound": {  
    "message": "No torrents found",  
    "description": "Empty state title"  
  },  
  "scanThisPage": {  
    "message": "Click refresh to scan this page",  
    "description": "Empty state subtitle"  
  },  
  "scanError": {  
    "message": "Scan error",  
    "description": "Error state title"  
  },  
  "tryAgain": {
```



```

    "message": "Click to try again",
    "description": "Error state subtitle"
  },
  "torrentsFound": {
    "message": "$count$ torrents found",
    "description": "Torrent count label",
    "placeholders": {
      "count": {
        "content": "$1",
        "example": "5"
      }
    }
  },
  "sendToBoba": {
    "message": "Send to Boba",
    "description": "Send button label"
  },
  "sendAllToBoba": {
    "message": "Send All to Boba",
    "description": "Send all button label"
  },
  "sendingTorrents": {
    "message": "Sending to Boba...",
    "description": "Sending state title"
  },
  "sendingItem": {
    "message": "$current$ of $total$: $name$",
    "description": "Current sending progress detail",
    "placeholders": {
      "current": { "content": "$1", "example": "1" },
      "total": { "content": "$2", "example": "5" },
      "name": { "content": "$3", "example": "filename.torrent" }
    }
  },
  "sentSuccessfully": {
    "message": "Sent successfully!",
    "description": "Success state title"
  },
  "sentDetail": {
    "message": "$count$ torrent(s) sent",
    "description": "Success detail text",
    "placeholders": {
      "count": { "content": "$1", "example": "3" }
    }
  },
  "viewInDashboard": {
    "message": "View in Dashboard",
    "description": "View dashboard button"
  },
  "openDashboard": {
    "message": "Dashboard",
    "description": "Dashboard link"
  }

```

```
},

"connecting": {
  "message": "Connecting...",
  "description": "Connection status"
},
"connected": {
  "message": "Connected",
  "description": "Connected status"
},
"notConnected": {
  "message": "Not connected",
  "description": "Disconnected status"
},
"serverError": {
  "message": "Server error",
  "description": "Server error status"
},
"scanning": {
  "message": "Scanning...",
  "description": "Scanning status"
},

"downloadStarted": {
  "message": "Download Started",
  "description": "Notification title"
},
"downloadComplete": {
  "message": "Download Complete",
  "description": "Notification title"
},
"downloadError": {
  "message": "Download Error",
  "description": "Notification title"
},
"downloading": {
  "message": "Downloading",
  "description": "Progress notification title"
},
"sentTitle": {
  "message": "Sent to Boba",
  "description": "Notification title"
},
"sentMessage": {
  "message": "$count$ torrent(s) sent to Boba",
  "description": "Notification message",
  "placeholders": {
    "count": { "content": "$1", "example": "3" }
  }
},
"sendFailed": {
  "message": "Failed to send",
```

```
    "description": "Error notification title"
  },
  "linkSent": {
    "message": "Link sent to Boba",
    "description": "Link sent confirmation"
  },
  "viewDashboard": {
    "message": "View Dashboard",
    "description": "Notification button"
  },
  "openFile": {
    "message": "Open File",
    "description": "Notification button"
  },
  "showFolder": {
    "message": "Show Folder",
    "description": "Notification button"
  },

  "optionsTitle": {
    "message": "Boba Settings",
    "description": "Options page title"
  },
  "serverConnection": {
    "message": "Server Connection",
    "description": "Settings section title"
  },
  "serverConnectionDesc": {
    "message": "Configure your Boba server URL and API
      credentials.",
    "description": "Settings section description"
  },
  "serverUrlLabel": {
    "message": "Boba Server URL",
    "description": "Settings label"
  },
  "serverUrlHelp": {
    "message":
      "The URL where your Boba server is running. Include the
      port if not using standard ports.",
    "description": "Settings help text"
  },
  "apiTokenLabel": {
    "message": "API Token (Optional)",
    "description": "Settings label"
  },
  "apiTokenHelp": {
    "message":
      "If your Boba server requires authentication, enter your
      API token here.",
    "description": "Settings help text"
  },
}
```

```
"testConnection": {
  "message": "Test Connection",
  "description": "Button label"
},
"downloadSettings": {
  "message": "Download Settings",
  "description": "Settings section title"
},
"defaultCategory": {
  "message": "Default Category",
  "description": "Settings label"
},
"categoryNone": {
  "message": "None",
  "description": "Category option"
},
"categoryMovies": {
  "message": "Movies",
  "description": "Category option"
},
"categoryTV": {
  "message": "TV Shows",
  "description": "Category option"
},
"categoryMusic": {
  "message": "Music",
  "description": "Category option"
},
"categorySoftware": {
  "message": "Software",
  "description": "Category option"
},
"categoryOther": {
  "message": "Other",
  "description": "Category option"
},
"autoStart": {
  "message": "Auto-start downloads",
  "description": "Settings label"
},
"autoStartHelp": {
  "message":
    "Automatically start downloading torrents when sent to  
Boba.",
  "description": "Settings help text"
},
"enableNotifications": {
  "message": "Enable notifications",
  "description": "Settings label"
},
"notificationsHelp": {
  "message": "Show browser notifications for download events.",
```

```
    "description": "Settings help text"
  },
  "behavior": {
    "message": "Behavior",
    "description": "Settings section title"
  },
  "scanMode": {
    "message": "Scan Mode",
    "description": "Settings label"
  },
  "scanManual": {
    "message": "Manual (click to scan)",
    "description": "Scan mode option"
  },
  "scanAuto": {
    "message": "Automatic (on page load)",
    "description": "Scan mode option"
  },
  "contextMenu": {
    "message": "Show context menu items",
    "description": "Settings label"
  },
  "dangerZone": {
    "message": "Danger Zone",
    "description": "Settings section title"
  },
  "resetSettings": {
    "message": "Reset All Settings",
    "description": "Button label"
  },
  "resetConfirm": {
    "message": "Reset all settings to defaults?",
    "description": "Confirmation dialog"
  },
  "saveSettings": {
    "message": "Save Settings",
    "description": "Button label"
  },
  "saved": {
    "message": "Saved!",
    "description": "Save status"
  },
  "saveFailed": {
    "message": "Save failed",
    "description": "Save error status"
  },
  "testing": {
    "message": "Testing...",
    "description": "Connection test status"
  },
  "connectionSuccess": {
```

```

    "message": "Connected!",
    "description": "Connection test result"
  },
  "connectionFailed": {
    "message": "Server error",
    "description": "Connection test result"
  },
  "connectionError": {
    "message": "Cannot connect",
    "description": "Connection test result"
  },
  "sidePanelTitle": {
    "message": "Boba Downloads",
    "description": "Side panel title"
  },
  "noActiveDownloads": {
    "message": "No active downloads",
    "description": "Empty state text"
  }
}

```

12.3 Manifest Configuration

```

{
  "default_locale": "en",
  "name": "__MSG_extName__",
  "description": "__MSG_extDescription__"
}

```

12.4 JavaScript i18n Usage

```

// In any extension page (popup, options, sidepanel):

// Get a localized string
const title = chrome.i18n.getMessage('popupTitle');

// With substitutions
const count = 5;
const message = chrome.i18n.getMessage('torrentsFound',
  [String(count)]);
// Result: "5 torrents found"

// Helper function for templates
function t(key, ...substitutions) {
  return chrome.i18n.getMessage(key, substitutions.map(String))
    || key;
}

// Apply to all data-i18n elements

```

```
document.querySelectorAll('[data-i18n]').forEach(el => {
  const key = el.dataset.i18n;
  const msg = chrome.i18n.getMessage(key);
  if (msg) el.textContent = msg;
});

// Get user's Accept-Languages
chrome.i18n.getAcceptLanguages((languages) => {
  console.log('User languages:', languages); // ['en-US', 'en',
  'es']
});

// Get current UI language
console.log(chrome.i18n.getUILanguage()); // 'en-US'
```

12.5 API Reference

Claim: i18n requires `_locales` directory with `messages.json` files. manifest MUST define "default_locale" if `_locales` exists. Strings referenced via `__MSG_key__` in manifest/CSS, `chrome.i18n.getMessage()` in JS. Supports placeholders up to 9 substitutions.

Source: Chrome Developer Docs

URL: <https://developer.chrome.com/docs/extensions/reference/api/i18n>

Date: 2026-05-09

Excerpt: "If an extension has a `_locales` directory, the manifest must define 'default_locale'... In JavaScript:

`chrome.i18n.getMessage('messagename')`"

Context: Fallback chain: user locale -> base locale (e.g., `en_GB` -> `en`) -> `default_locale`

Confidence: high

Claim: Predefined messages available: `@@ui_locale`, `@@bidi_dir`, `@@bidi_reversed_dir`, `@@bidi_start_edge`, `@@bidi_end_edge`. Useful for RTL support.

Source: Chromium i18n Documentation

URL: <https://www.chromium.org/developers/design-documents/extensions/how-the-extension-system-works/i18n/>

Date: N/A

Excerpt: "`chrome.i18n.getMessage('@@bidi_dir')` returns '`ltr`' or '`rtl`'"

Context: Use for RTL layout adjustments

Confidence: high

Appendix A: Complete manifest.json

```
{
  "manifest_version": 3,
```

```
"name": "__MSG_extName__",
"description": "__MSG_extDescription__",
"version": "1.0.0",
"default_locale": "en",

"permissions": [
  "storage",
  "activeTab",
  "scripting",
  "contextMenus",
  "notifications",
  "sidePanel"
],

"host_permissions": [
  "http://localhost:*/",
  "https://*/"
],

"action": {
  "default_icon": {
    "16": "icons/icon-16.png",
    "32": "icons/icon-32.png",
    "48": "icons/icon-48.png",
    "128": "icons/icon-128.png"
  },
  "default_title": "Boba Torrent Helper",
  "default_popup": "popup/popup.html"
},

"background": {
  "service_worker": "background/service-worker.js",
  "type": "module"
},

"options_page": "options/options.html",

"side_panel": {
  "default_path": "sidepanel/sidepanel.html"
},

"commands": {
  "send-all-torrents": {
    "suggested_key": {
      "default": "Ctrl+Shift+B",
      "mac": "Command+Shift+B"
    },
    "description": "Send all detected torrents to Boba"
  },
  "scan-page": {
    "suggested_key": {
      "default": "Ctrl+Shift+S",
```



```

        "mac": "Command+Shift+S"
    },
    "description": "Scan current page for torrents"
},
"open-dashboard": {
    "suggested_key": {
        "default": "Ctrl+Shift+D",
        "mac": "Command+Shift+D"
    },
    "description": "Open Boba dashboard"
},
"toggle-side-panel": {
    "suggested_key": {
        "default": "Ctrl+Shift+P",
        "mac": "Command+Shift+P"
    },
    "description": "Toggle Boba side panel"
}
},

"icons": {
    "16": "icons/icon-16.png",
    "32": "icons/icon-32.png",
    "48": "icons/icon-48.png",
    "128": "icons/icon-128.png"
},

"web_accessible_resources": [
    {
        "resources": ["icons/*"],
        "matches": ["<all_urls>"]
    }
]
}

```

Appendix B: File Structure

```

boba-extension/
  manifest.json

```

```

  _locales/
    en/
      messages.json

```

```

background/
  service-worker.js      # Main service worker entry
  contextMenus.js        # Context menu registration
  iconState.js           # Badge/icon state management
  notifications.js       # Notification handlers

```

commands.js	# Keyboard shortcut handlers
sidePanel.js	# Side panel configuration
popup/	
popup.html	# Popup HTML
popup.css	# Popup styles (theme-aware)
popup.js	# Popup logic
options/	
options.html	# Settings page HTML
options.css	# Settings styles
options.js	# Settings logic
sidepanel/	
sidepanel.html	# Side panel HTML
sidepanel.css	# Side panel styles
sidepanel.js	# Side panel logic
icons/	
icon-16.png	# Toolbar icon
icon-32.png	# Toolbar @2x
icon-48.png	# Chrome Web Store
icon-128.png	# Store listing
icon-success-128.png	# Success notification
icon-error-128.png	# Error notification
utils/	
theme.js	# Theme detection utilities
ally.js	# Accessibility utilities
i18n.js	# i18n helper functions

Appendix C: Key API Version Reference

API	Min Chrome	MV Version	Notes
chrome.action	88	MV3	Replaces browserAction/ pageAction
chrome.storage.session	102	MV3	In-memory session storage
chrome.sidePanel	114	MV3	Side panel for persistent UI
chrome.sidePanel.open()	116	MV3	Programmatic open
chrome.sidePanel.close()	141	MV3	Programmatic close
chrome.notifications	28		Rich notifications

API	Min Chrome	MV Version	Notes
		MV2/ MV3	
<code>chrome.contextMenus</code>	6	MV2/ MV3	Right-click menus
<code>chrome.commands</code>	35	MV2/ MV3	Keyboard shortcuts
<code>chrome.i18n</code>	1	MV2/ MV3	Internationalization
<code>chrome.storage.sync</code>	1	MV2/ MV3	Cross-device sync (100KB)
<code>chrome.storage.local</code>	1	MV2/ MV3	Local storage (10MB)

Research compiled from 20+ authoritative sources including official Chrome Developer documentation, MDN Web Docs, Chromium source code, and established extension development guides. All code examples are written for Manifest V3 and target Chrome 114+.